

Manuel NSI Première

Édition de travail 2026-2027

Table des matières

1	Représentation des données : types et valeurs de base	1
1.1	Écriture d'un entier dans une base	1
1.2	Représentation binaire des entiers relatifs	2
1.3	Représentation approximative des nombres flottants	3
1.4	Valeurs et opérateurs booléens	4
1.5	Représentation d'un texte en machine	5
2	Types construits : p-uplets, tableaux, dictionnaires	9
2.1	Les p-uplets (tuples)	9
2.2	Les tableaux (listes Python)	10
2.3	Les grilles : tableaux de tableaux	12
2.4	Les dictionnaires	13
2.5	Mutabilité et références	15
3	Traitement de données en tables	41
3.1	Importer une table	41
3.2	Recherche dans une table	42
3.3	Trier une table	44
3.4	Fusionner deux tables	45
4	Langages et programmation	49
4.1	Constructions élémentaires	49
4.2	Diversité et unité des langages de programmation	50
4.3	Spécifier une fonction	51
4.4	Mise au point de programmes	52
4.5	Utiliser une bibliothèque	53
5	Algorithmique 1 : parcours et tris	57
5.1	Parcours séquentiel d'un tableau	57
5.2	Le tri par insertion	58
5.3	Le tri par sélection	59
6	Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins	63
6.1	Recherche dichotomique	63
6.2	Algorithmes gloutons	64
6.3	L'algorithme des k plus proches voisins	65
7	Interactions entre l'homme et la machine sur le Web	69
7.1	Composants graphiques et événements	69
7.2	Interaction client-serveur	70
7.3	Formulaires, requêtes GET et POST	72
8	Architecture de von Neumann et systèmes d'exploitation	76
8.1	L'architecture de von Neumann	76
8.2	Dérouler l'exécution d'instructions machine	77
8.3	Systèmes d'exploitation	78
8.4	La ligne de commande	79
8.5	Droits et permissions d'accès	79
9	Réseaux : transmission de données	83
9.1	Découpage en paquets et encapsulation	83
9.2	Le protocole du bit alterné	84

9.3	Simuler un réseau	85
9.4	Capteurs, actionneurs et IHM programmée	86
10	Projet et méthodes	90
10.1	Repères d'histoire de l'informatique	90
10.2	La démarche de projet	91
10.3	Déboguer méthodiquement	92
10.4	Documenter son code et préparer une présentation orale	93

1 Représentation des données : types et valeurs de base

1.1 Écriture d'un entier dans une base

DÉFINITION D1

La **base** b (avec $b \geq 2$) d'un système de numération est le nombre de chiffres distincts utilisés pour écrire un nombre. En base b , un nombre s'écrit $\overline{d_k d_{k-1} \dots d_1 d_0}_b$ et sa valeur en base 10 est $\sum_{i=0}^k d_i \times b^i$.

| Une machine ne manipule que des 0 et des 1 : le *bit*. Toute donnée est finalement une suite de bits, dont le sens dépend du codage choisi.

Exemple. En base 2 (*binaire*, chiffres 0 et 1), $101101_2 = 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 45$. En base 16 (*hexadécimal*, chiffres 0-9 puis A-F), $2D_{16} = 2 \times 16 + 13 = 45$.

Propriété Conversion base b vers base 10. Pour convertir un nombre écrit en base b vers la base 10, on calcule la somme des chiffres multipliés par les puissances de b correspondantes (méthode dite « d'Horner » pour un calcul efficace : on part du chiffre de poids fort et on accumule $r \leftarrow r \times b + d_i$).

CODE DE RÉFÉRENCE — Conversion base b vers décimal

```
1 def vers_decimal(chiffres, base):
2     """Convertit une liste de chiffres (poids fort en premier) en un entier
3     decimal.
4
5     Preconditions : base ≥ 2, chaque chiffre de 'chiffres' est dans [0, base[.
6     """
7     valeur = 0
8     for d in chiffres:
9         valeur = valeur * base + d
10    return valeur
11
12 print(vers_decimal([1, 0, 1, 1, 0, 1], 2)) # 45
13 print(vers_decimal([2, 13], 16)) # 45
```

Propriété Conversion base 10 vers base b . Pour convertir un entier positif de la base 10 vers la base b , on effectue des divisions euclidiennes successives par b : les restes obtenus, lus de bas en haut (dernier reste calculé en premier), forment l'écriture en base b .

Exemple. Pour convertir 45 en base 2 : $45 = 2 \times 22 + 1$, $22 = 2 \times 11 + 0$, $11 = 2 \times 5 + 1$, $5 = 2 \times 2 + 1$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : 101101_2 .

ERREUR FRÉQUENTE

Confondre la **valeur** d'un nombre et son **écriture**. Le nombre quarante-cinq est le même objet mathématique, qu'on l'écrive 101101 en base 2, 2D en base 16 ou 45 en base 10 : seule son écriture change selon la base.

» **Python À la machine.** Écris une fonction `vers_base(n, base)` qui renvoie la liste des chiffres de l'entier positif n écrit en base `base` (poids fort en premier), puis vérifie que `vers_base(255, 16)` donne `[15, 15]`. Compare avec les fonctions natives `bin`, `oct` et `hex` de Python.

1.2 Représentation binaire des entiers relatifs

DÉFINITION D2

Un entier écrit sur n **bits** peut représenter 2^n valeurs distinctes. Pour un entier **positif**, ces valeurs vont de 0 à $2^n - 1$: sur 8 bits, de 0 à 255.

Propriété Nombre de bits nécessaires. Le nombre de bits nécessaires pour écrire un entier positif n (avec $n \geq 1$) est $\lfloor \log_2(n) \rfloor + 1$. En pratique, on utilise des tailles standard : 8, 16, 32 ou 64 bits (Python, lui, ajuste automatiquement la taille de ses entiers, sans limite fixe).

Exemple. 200 s'écrit sur 8 bits ($200 < 256 = 2^8$), mais $200 + 150 = 350$ nécessite 9 bits ($350 \geq 256$ mais $350 < 512 = 2^9$) : une addition peut faire « déborder » le nombre de bits disponibles.

DÉFINITION D3

Le **complément à 2** est la méthode standard pour représenter les entiers **relatifs** (positifs et négatifs) en binaire, sur un nombre fixe n de bits. Pour représenter $-x$ (avec $x > 0$) : on écrit x en binaire sur n bits, on inverse chaque bit (0 devient 1, 1 devient 0), puis on ajoute 1 au résultat.

Exemple. Sur 8 bits, pour représenter -5 : 5 s'écrit 00000101 ; en inversant chaque bit on obtient 11111010 ; en ajoutant 1 on obtient 11111011, qui est la représentation de -5 en complément à 2 sur 8 bits.

CODE DE RÉFÉRENCE — Complément à 2 sur n bits

```
1 def complement_a_2(x, n):
2     """Renvoie l'écriture sur n bits de l'entier relatif x (complement a 2).
3
4     Preconditions : -2**(n-1) ≤ x < 2**(n-1).
5     """
6     if x < 0:
7         x = (1 << n) + x # equivaut a l'inversion des bits puis +1
8     return format(x, f'0{n}b')
9
10 print(complement_a_2(5, 8)) # 00000101
11 print(complement_a_2(-5, 8)) # 11111011
12 print(complement_a_2(-1, 8)) # 11111111
```

Propriété Intérêt du complément à 2. Le complément à 2 permet d'additionner deux entiers relatifs **avec le même circuit binaire** que pour des entiers positifs, sans traitement particulier du signe : le bit de poids fort indique le signe (0 = positif, 1 = négatif), et l'addition « déborde » naturellement de façon cohérente.

ERREUR FRÉQUENTE

Croire qu'un nombre négatif se code en changeant simplement le bit de poids fort (« signe + valeur absolue »). Ce n'est **pas** le complément à 2 : cette méthode naïve poserait des problèmes pour l'addition (deux écritures de 0) et n'est pas celle utilisée par les machines réelles.

» **Python À la machine.** Vérifie avec la fonction `complement_a_2(x, 8)` ci-dessus l'écriture de -1 , -128 et 127 sur 8 bits. Que se passe-t-il si tu essaies d'appeler la fonction avec $x = -129$ ou $x = 128$? Pourquoi (relie ta réponse à la précondition indiquée dans la docstring) ?

1.3 Représentation approximative des nombres flottants

DÉFINITION D4

Un **nombre flottant** est une valeur numérique approchée d'un nombre réel, stockée sur un nombre fixe de bits. Beaucoup de nombres qui s'écrivent simplement en base 10 (comme 0,1) n'ont **pas** d'écriture binaire finie, exactement comme $1/3 = 0,333 \dots$ n'a pas d'écriture décimale finie.

Exemple. En Python, 0.1 n'est pas stocké exactement : c'est une valeur très proche mais légèrement différente. La conséquence la plus visible :

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Propriété Ne jamais tester l'égalité de deux flottants. Comparer deux flottants avec `==` est une erreur classique : à cause des erreurs d'arrondi, deux calculs mathématiquement égaux peuvent donner des valeurs flottantes légèrement différentes. Il faut comparer leur **écart** à une petite tolérance près.

CODE DE RÉFÉRENCE — Comparaison de flottants avec tolérance

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     """Teste si deux flottants sont égaux à une tolerance pres."""
3     return abs(a - b) < tolerance
4
5 print(0.1 + 0.2 == 0.3)           # False (piege classique)
6 print(presque_egaux(0.1 + 0.2, 0.3)) # True
```

| Un nombre flottant (float) représente un nombre réel avec une précision limitée. La plupart des nombres réels n'ont pas d'écriture binaire finie, tout comme $1/3$ n'a pas d'écriture décimale finie.

ERREUR FRÉQUENTE

Écrire `if resultat == valeur_attendue:` pour tester un calcul flottant (EF). Cette comparaison peut échouer même quand le calcul est correct, à cause des erreurs d'arrondi : il faut toujours comparer avec une tolérance, via une fonction comme `presque_egaux` ci-dessus.

POUR ALLER PLUS LOIN

La norme utilisée par la quasi-totalité des langages (dont Python) est IEEE-754, qui code un flottant en trois parties : un bit de signe, un exposant, et une mantisse (partie fractionnaire). Aucune connaissance précise de cette norme n'est exigible au programme, mais elle explique pourquoi $1/3$, $0,1$ ou $0,2$ ne sont pas représentables exactement en base 2 (seules les fractions dont le dénominateur est une puissance de 2, comme $0,25 = 1/4$, le sont).

Exemple. $0,25 = 1/4 = 2^{-2}$ est représentable **exactement** en binaire, contrairement à $0,1$:

» **Python À la machine.** Dans un éditeur Python, calcule $0.1 + 0.1 + 0.1$ et compare-le à 0.3 avec `==`, puis avec la fonction `presque_egaux` ci-dessus. Cherche un autre triplet de flottants dont la somme affiche un résultat « bizarre » comme `0.30000000000000004`.

1.4 Valeurs et opérateurs booléens

DÉFINITION D5

Une **valeur booléenne** ne prend que deux valeurs : `True` (vrai, souvent codé 1) ou `False` (faux, souvent codé 0). Les trois opérateurs booléens de base sont `and` (et), `or` (ou) et `not` (non).

Propriété Table de vérité de `and` et `or`.

<i>a</i>	<i>b</i>	<i>a and b</i>	<i>a or b</i>
Vrai	Vrai	Vrai	Vrai
Vrai	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai
Faux	Faux	Faux	Faux

`and` est vrai seulement si **les deux** opérandes sont vrais ; `or` est vrai dès qu'**au moins un** des deux opérandes est vrai.

DÉFINITION D6

Le **ou exclusif** (xor, noté $\oplus$) est vrai lorsque **exactement un** des deux opérandes est vrai (mais pas les deux). En Python, on l'obtient avec `a != b` pour des booléens, ou l'opérateur `^` bit à bit sur des entiers.

Exemple. Le xor sert par exemple à additionner deux bits sans retenue : $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$ (la retenue, elle, correspond à `a and b`).

CODE DE RÉFÉRENCE — Addition d'un bit (demi-additionneur)

```
1 def demi_additionneur(a, b):
2     """Additionne deux bits (0 ou 1) : renvoie (somme, retenue)."""
```



```

3     somme = a ^ b
4     retenue = a & b
5     return somme, retenue
6
7 print(demi_additionneur(1, 1)) # (0, 1) : 1+1 = 10 en binaire
8 print(demi_additionneur(1, 0)) # (1, 0)

```

Propriété Table de vérité d'une expression composée. Pour dresser la table de vérité d'une expression booléenne à n variables, on énumère les 2^n combinaisons possibles de valeurs des variables, et on calcule la valeur de l'expression pour chacune.

Exemple. Table de vérité de $(a \text{ and } b) \text{ or } (\text{not } a \text{ and } c)$:

ERREUR FRÉQUENTE

Croire que `and` et `or` évaluent toujours leurs deux opérandes. En Python, l'évaluation est **séquentielle** et **court-circuitée** : dans `a and b`, si `a` vaut `False`, `b` n'est même pas évalué (le résultat est déjà déterminé). Cela peut avoir des effets de bord si `b` contient un appel de fonction.

» **Python À la machine.** Écris une fonction `table_verite(expr, n)` qui... en fait, dresse « à la main » (avec des boucles `for`) la table de vérité de l'expression $(a \text{ or } b) \text{ and } (\text{not } c)$ pour les 8 combinaisons de `a`, `b`, `c`. Vérifie ton résultat avec Python.

1.5 Représentation d'un texte en machine

DÉFINITION D7

Un **encodage de caractères** est une correspondance entre des caractères (lettres, chiffres, symboles) et des suites de bits. Coder un texte, c'est appliquer cette correspondance à chacun de ses caractères.

Propriété Trois encodages historiques.

- ▶ **ASCII** (1963) : code chaque caractère sur 7 bits (128 caractères) : lettres non accentuées, chiffres, ponctuation de base. Ne code pas les caractères accentués (é, à, ç...).
- ▶ **ISO-8859-1** (« Latin-1 ») : étend ASCII à 8 bits (256 caractères), en ajoutant les caractères accentués usuels des langues d'Europe de l'Ouest.
- ▶ **Unicode** (via l'encodage **UTF-8**) : vise à coder **tous** les caractères de toutes les langues (et les emojis). Chaque caractère occupe de 1 à 4 octets selon sa position dans la table Unicode.

Exemple. Le caractère 'A' a pour code 65 dans ASCII, ISO-8859-1 et UTF-8 (les 128 premiers caractères Unicode coïncident avec ASCII). Le caractère 'é' n'existe pas en ASCII, vaut 233 en ISO-8859-1 (un seul octet), et occupe **deux** octets en UTF-8.

Contre-exemple. Tenter d'encoder un caractère accentué en ASCII provoque une erreur, car ce caractère n'existe pas dans cet encodage :

```

»> 'café'.encode('ascii')
UnicodeEncodeError: 'ascii' codec can't encode character '\xe9' in position 3:

```

```
ordinal not in range(128)
```

CODE DE RÉFÉRENCE — Convertir un texte d'un encodage à un autre

```
1 def reencoder(texte, encodage_source, encodage_cible):
2     """Reencode un texte : decode avec la source puis encode avec la cible.
3
4     Preconditions : 'texte' est deja une chaine Python (str), pas des octets.
5     """
6     octets = texte.encode(encodage_source)
7     return octets.decode(encodage_source).encode(encodage_cible)
8
9 octets_latin1 = reencoder('café', 'utf-8', 'iso-8859-1')
10 print(octets_latin1)           # b'caf\xe9'
11 print(len(octets_latin1))      # 4 octets (contre 5 en UTF-8)
```

Propriété Intérêt d'un encodage universel. Unicode/UTF-8 s'est imposé car il permet d'échanger des textes entre systèmes utilisant des langues différentes (et donc des jeux de caractères différents) sans ambiguïté ni perte d'information, contrairement à ASCII (trop limité) ou aux nombreux encodages régionaux incompatibles entre eux (ISO-8859-1 pour l'Europe de l'Ouest, d'autres pour le cyrillique, l'arabe, etc.).

ERREUR FRÉQUENTE

Confondre le **nombre de caractères** d'une chaîne et son **nombre d'octets** une fois encodée. `len('café')` vaut 4 (quatre caractères), mais `len('café'.encode('utf-8'))` vaut 5 (le « é » occupe deux octets en UTF-8).

» **Python À la machine.** Dans un éditeur Python, encode la chaîne "NSI 2026 – à bientôt !" en UTF-8 puis en ISO-8859-1, et compare le nombre d'octets obtenus avec `len()`. Essaie ensuite de l'encoder en ASCII : observe et explique l'erreur.

Exercice 1 10 min

1. Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
2. Convertir 11001_2 en base 10.
3. Convertir $3F_{16}$ en base 10.

Exercice 2 12 min

1. Combien de bits sont nécessaires pour écrire 300 en binaire ?
2. Écrire 20 en complément à 2 sur 8 bits.
3. Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 3 12 min

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10 € font bien 0,30 € :

```

1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")

```

1. Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?
2. Écrire une fonction `presque_egaux(a, b, tolerance=1e-9)` qui teste si deux flottants sont égaux à une tolérance près.
3. Réécrire le test `if sum(prix) == 0.30:` en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 4 ♦♦

⌚ 12 min

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

1. Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
2. Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
3. En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 5 ♦♦♦

On considère la chaîne "Bonjour à tous !".

1. Combien cette chaîne compte-t-elle de **caractères** ?
2. Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
3. Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
4. Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

QCM d'auto-évaluation — Types et valeurs de base

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Écriture dans une base

1. [Q1] L'écriture 1010_2 correspond, en base 10, à :
 - A. 8
 - B. 10
 - C. 12
 - D. 1010

C2 — Complément à 2

2. [Q2] Sur 8 bits, combien de valeurs entières distinctes peut-on représenter ?
 - A. 8
 - B. 128

- C. 255
- D. 256

3. [Q3] En complément à 2, le bit de poids fort indique :
- A. la parité du nombre.
 - B. le signe du nombre.
 - C. le nombre de bits utilisés.
 - D. rien de particulier.

C3 — Flottants

4. [Q4] L'expression $0.1 + 0.2 == 0.3$ en Python renvoie :
- A. True, car $0,1 + 0,2 = 0,3$.
 - B. False, à cause des erreurs d'arrondi de la représentation flottante.
 - C. Une erreur de syntaxe.
 - D. Cela dépend de la version de Python.

C4 — Booléens

5. [Q5] L'expression `True and False` vaut :
- A. True
 - B. False
 - C. None
 - D. Une erreur

C5 — Encodage de texte

6. [Q6] Un texte contenant uniquement des lettres non accentuées et des chiffres peut être encodé sans erreur :
- A. uniquement en UTF-8.
 - B. uniquement en ISO-8859-1.
 - C. en ASCII, en ISO-8859-1 et en UTF-8.
 - D. dans aucun de ces trois encodages.

2 Types construits : p-uplets, tableaux, dictionnaires

2.1 Les p-uplets (tuples)

DÉFINITION D1

Un **p-uplet** (ou *tuple*) est une séquence ordonnée de p éléments, éventuellement de types différents, séparés par des virgules et entourés de parenthèses. Un p-uplet est **non mutable** : une fois créé, on ne peut ni ajouter, ni retirer, ni modifier ses éléments.

| Un p-uplet est une collection *ordonnée* et *non modifiable* de valeurs. En Python, on utilise le mot anglais *tuple*.

Exemple. Le tuple `station = ("Tunis–Carthage", 36.8, 10.2)` contient trois éléments : une chaîne de caractères et deux flottants. On accède à chaque élément par son indice, en commençant à 0 :

```
1 station = ("Tunis-Carthage", 36.8, 10.2)
2 nom = station[0]           # "Tunis-Carthage"
3 latitude = station[1]      # 36.8
4 longitude = station[2]     # 10.2
```

Contre-exemple. Toute tentative de modification provoque une erreur :

```
>>> station[1] = 37.0
TypeError: 'tuple' object does not support item assignment
```

ERREUR FRÉQUENTE

Modifier un tuple comme une liste (EF1). Un tuple est immuable par conception : si l'on a besoin de modifier des données, il faut utiliser une liste.

DÉFINITION D2

La fonction `len(t)` renvoie le nombre d'éléments d'un tuple `t`. L'opérateur `in` teste l'appartenance d'une valeur à un tuple.

Exemple. Vérifier qu'une station est dans une zone :

```
1 coordonnees = (36.8, 10.2)
2 print(len(coordonnees))    # 2
3 print(36.8 in coordonnees) # True
```

DÉFINITION D3

Une fonction peut renvoyer un tuple pour communiquer *plusieurs résultats* à la fois. On parle de **déballage** (*unpacking*) lorsqu'on affecte chaque élément du tuple à une variable distincte.

CODE DE RÉFÉRENCE — Fonction renvoyant un tuple

```

1 def milieu(a, b):
2     """Renvoie le milieu de deux points (tuples de coordonnées).
3
4     Préconditions : a et b sont des tuples de 2 flottants.
5     """
6     mx = (a[0] + b[0]) / 2
7     my = (a[1] + b[1]) / 2
8     return (mx, my)
9
10 # Déballage du résultat
11 x, y = milieu((1, 2), (5, 8))
12 print(x, y) # 3.0 5.0

```

Le déballage est très utilisé en Python. On écrit `x, y = milieu(a, b)` plutôt que `r = milieu(a, b); x = r[0]; y = r[1]`.

POUR ALLER PLUS LOIN

Un tuple de longueur 1 s'écrit `(42,)` avec une virgule obligatoire. Sans virgule, `(42)` est simplement l'entier 42 entre parenthèses. Le tuple vide s'écrit `()`.

» **Python À la machine.** Ouvre un éditeur Python et définis un tuple `eleve` contenant un nom, un âge et une moyenne. Accède à chaque élément par son indice. Essaie de modifier l'âge : observe l'erreur. Écris une fonction `presente(eleve)` qui renvoie une chaîne du type "Ali, 16 ans, moyenne 14.5".

2.2 Les tableaux (listes Python)

En Python, se représente une liste (list). au tuple mutable modifier

DÉFINITION D4

Un **tableau** est une séquence ordonnée d'éléments accessibles par leur indice, en commençant à 0. En Python, il est représenté par le type `list`, délimité par des crochets. Un tableau est **mutable** : on peut modifier, ajouter ou supprimer des éléments.

Exemple. Créer et manipuler un tableau de relevés de températures :

```

1 releves = [18, 20, 19, 22, 17]
2 print(releves[0]) # 18 (premier élément)
3 print(releves[-1]) # 17 (dernier élément)
4 print(len(releves)) # 5
5 releves[2] = 21 # modification autorisée
6 print(releves) # [18, 20, 21, 22, 17]

```

Contre-exemple. Accéder à un indice inexistant provoque une erreur :

```

>>> releves = [18, 20, 19]
>>> releves[3]
IndexError: list index out of range

```

ERREUR FRÉQUENTE

Indice hors limites (off-by-one). Un tableau de n éléments a des indices de 0 à $n - 1$. L'indice n n'existe pas.

DÉFINITION D5

Les méthodes principales d'une liste :

- ▶ `t.append(v)` : ajoute v en fin de liste.
- ▶ `t.pop()` : supprime et renvoie le dernier élément.
- ▶ `t.insert(i, v)` : insère v à l'indice i .
- ▶ `v in t` : teste si v est dans t .

CODE DE RÉFÉRENCE — Parcours d'un tableau

```
1 releves = [18, 20, 19, 22, 17]
2
3 # Parcours par valeur (préférez quand l'indice est inutile)
4 for temperature in releves:
5     print(temperature)
6
7 # Parcours par indice (quand on a besoin de la position)
8 for i in range(len(releves)):
9     print(f"Relevé {i} : {releves[i]}")
```

ERREUR FRÉQUENTE

Parcourir les indices quand les valeurs suffisent (EF2). Écrire `for i in range(len(t)): print(t[i])` quand `for v in t: print(v)` suffit. Le parcours par valeur est plus lisible et moins sujet aux erreurs d'indice.

DÉFINITION D6

Un **tableau en compréhension** se construit en une seule expression : `[expression for variable in itérable]` avec un filtre optionnel `if condition`.

Exemple. Construire un tableau des carrés des entiers de 0 à 9 :

```
1 carres = [x ** 2 for x in range(10)]
2 print(carres) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
3
4 # Avec filtre : garder uniquement les carrés pairs
5 pairs = [x ** 2 for x in range(10) if x % 2 == 0]
6 print(pairs) # [0, 4, 16, 36, 64]
```

CODE DE RÉFÉRENCE — Moyenne d'un tableau

```
1 def moyenne(notes):
2     """Renvoie la moyenne d'un tableau de nombres.
```

La compréhension remplace avantageusement une boucle `for` avec `append` : elle est plus concise et plus rapide.

```

3
4     Précondition : notes est un tableau non vide de nombres.
5     Lève ValueError si le tableau est vide.
6     """
7     if len(notes) == 0:
8         raise ValueError("le tableau est vide")
9     return sum(notes) / len(notes)
10
11 print(moyenne([12, 14, 16])) # 14.0

```

» **Python À la machine.** Crée un tableau prenomns contenant cinq prénoms. Ajoute un prénom avec append, retire le dernier avec pop. Construis en compréhension le tableau des longueurs de chaque prénom. Affiche la longueur moyenne.

2.3 Les grilles : tableaux de tableaux

| Une grille (ou matrice) se représente en Python par un tableau dont chaque élément est lui-même un tableau. On parle de *tableau à deux dimensions*.

DÉFINITION D7

Une **grille** est un tableau de tableaux où chaque sous-tableau représente une ligne. On accède à l'élément de la ligne *i* et de la colonne *j* par `grille[i][j]`.

Exemple. Une grille de pixels en niveaux de gris (0 = noir, 255 = blanc) :

```

1 image = [
2     [0, 0, 0],
3     [0, 255, 0],
4     [0, 0, 0],
5 ]
6 print(image[1][1]) # 255 (pixel central)

```

	0	1	2
ligne 0	0	0	0
ligne 1	0	255	0
ligne 2	0	0	0

`image[1][1]`

CODE DE RÉFÉRENCE — Parcours d'une grille

```

1 def afficher_grille(grille):
2     """Affiche une grille ligne par ligne."""
3     for ligne in grille:
4         for valeur in ligne:
5             print(f"{valeur:4d}", end=" ")
6         print()
7
8 def nb_lignes(grille):
9     """Renvoie le nombre de lignes."""
10    return len(grille)
11
12 def nb_colonnes(grille):
13    """Renvoie le nombre de colonnes (suppose grille non vide)."""
14    return len(grille[0])

```


ERREUR FRÉQUENTE

Confondre lignes et colonnes. Dans `grille[i][j]`, `i` est le numéro de *ligne* et `j` le numéro de *colonne*.

DÉFINITION D8

Pour construire une grille $n \times m$ remplie de zéros, on utilise une **compréhension imbriquée** : `[[0 for j in range(m)] for i in range(n)]`.

Contre-exemple. Ne jamais écrire `[[0] * m] * n` : toutes les lignes seraient des *alias* du même objet (voir section 2.5).

```
1 # CORRECT : chaque ligne est un objet distinct
2 grille = [[0 for j in range(3)] for i in range(3)]
3 grille[0][0] = 1
4 print(grille) # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
5
6 # INCORRECT : alias dangereux
7 mauvaise = [[0] * 3] * 3
8 mauvaise[0][0] = 1
9 print(mauvaise) # [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

» **Python À la machine.** Construis une grille 4x4 de zéros en compréhension. Place un 1 sur la diagonale (`grille[i][i] = 1` pour chaque `i`). Affiche le résultat et vérifie que tu obtiens la matrice identité.

2.4 Les dictionnaires

DÉFINITION D9

Un **dictionnaire** est une collection non ordonnée de paires *clé : valeur*, délimitée par des accolades. Les clés sont uniques et non mutables (chaîne, entier, tuple). Les valeurs peuvent être de n'importe quel type. Un dictionnaire est **mutable**.

Exemple. Un dictionnaire de mesures météorologiques :

```
1 mesures = {"temperature": 21, "vent": 12, "humidite": 65}
2 print(mesures["temperature"]) # 21
```

Contre-exemple. Accéder à une clé inexistante provoque une erreur :

```
>> mesures["pression"]
KeyError: 'pression'
```

ERREUR FRÉQUENTE

Accéder à une clé sans vérifier sa présence (EF3). Toujours tester avec `if cle in dico` ou utiliser `dico.get(cle, default)` qui renvoie la valeur par défaut si la clé est absente.

Un dictionnaire associe des clés à des valeurs. Contrairement à un tableau, on n'accède pas aux éléments par un indice numérique, mais par une clé (souvent une chaîne de caractères).

DÉFINITION D10

Opérations principales sur un dictionnaire d :

- ▶ `d[cle] = valeur` : ajoute ou modifie une entrée.
- ▶ `del d[cle]` : supprime une entrée.
- ▶ `cle in d` : teste la présence d'une clé.
- ▶ `len(d)` : nombre de paires.
- ▶ `d.get(cle, default)` : accès sécurisé.

CODE DE RÉFÉRENCE — Parcours d'un dictionnaire

```
1 mesures = {"temperature": 21, "vent": 12, "humidite": 65}
2
3 # Parcours des clés
4 for cle in mesures:
5     print(cle, ":", mesures[cle])
6
7 # Parcours des paires clé-valeur (préférée)
8 for cle, valeur in mesures.items():
9     print(f"{cle} = {valeur}")
```

Le parcours par `.items()` est plus lisible et plus efficace que d'accéder à `dico[cle]` dans la boucle.

CODE DE RÉFÉRENCE — Construire un dictionnaire depuis des données

```
1 def stations_chaudes(stations, seuil):
2     """Renvoie les noms des stations dont la température
3     dépasse le seuil.
4
5     Précondition : stations est une liste de dictionnaires
6     avec les clés 'nom' et 'temperature'.
7     """
8     resultat = []
9     for s in stations:
10         if s["temperature"] > seuil:
11             resultat.append(s["nom"])
12     return resultat
13
14 stations = [
15     {"nom": "Nord", "temperature": 18},
16     {"nom": "Sud", "temperature": 24},
17     {"nom": "Est", "temperature": 20},
18 ]
19 print(stations_chaudes(stations, 17)) # ["Nord", "Sud", "Est"]
```

POUR ALLER PLUS LOIN

Un dictionnaire en compréhension s'écrit `{cle: valeur for cle, valeur in iterable}`. Par exemple, `{s["nom"]: s["temperature"] for s in stations}` produit `{"Nord": 18, "Sud": 24, "Est": 20}`.

» **Python À la machine.** Crée un dictionnaire `eleve` contenant les clés `"nom"`, `"classe"` et `"notes"` (cette dernière étant un tableau de nombres). Ajoute une clé `"moyenne"` calculée à partir des notes. Affiche toutes les informations avec une boucle sur `.items()`.

2.5 Mutabilité et références

DÉFINITION D11

Lorsqu'on écrit `b = a` pour un objet mutable, `b` ne contient pas une copie de `a` : les deux variables désignent **le même objet** en mémoire. On dit que `b` est un **alias** de `a`. Toute modification via l'une des variables est visible via l'autre.

Exemple. L'affectation crée un alias, pas une copie :

```
1 notes = [12, 15, 9]
2 copie = notes          # alias : même objet
3 copie.append(18)
4 print(notes)           # [12, 15, 9, 18]
5 print(len(copie))      # 4
```



ERREUR FRÉQUENTE

Croire que `copie = notes` crée une copie indépendante. C'est l'erreur la plus fréquente en NSI sur les types mutables.

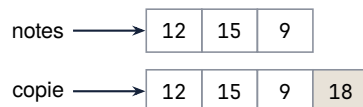
DÉFINITION D12

Pour obtenir une **copie indépendante** d'une liste, on utilise :

- ▶ `copie = list(original)` ou `copie = original[:]` (copie *superficielle*).
- ▶ `copie = [list(ligne) for ligne in original]` pour une grille (copie de chaque sous-liste).

Exemple. Copie indépendante :

```
1 notes = [12, 15, 9]
2 copie = list(notes)    # copie indépendante
3 copie.append(18)
4 print(notes)           # [12, 15, 9] (inchangé)
5 print(copie)           # [12, 15, 9, 18]
```



Contre-exemple. Copie superficielle insuffisante pour les listes imbriquées :

```
1 grille = [[1, 2], [3, 4]]
2 copie = list(grille)    # copie superficielle
3 copie[0][0] = 99
4 print(grille)           # [[99, 2], [3, 4]]; modifié !
```

ERREUR FRÉQUENTE

Copier une liste imbriquée seulement au premier niveau (EF4). Avec `list(grille)`, les sous-listes restent partagées. Pour une copie profonde, copier chaque sous-liste : `[list(ligne) for ligne in grille]`.

CODE DE RÉFÉRENCE — Effet de bord dans une fonction

```
1 def ajouter_note(bulletin, note):
2     """Ajoute une note au bulletin.
3
4     ATTENTION : modifie le bulletin passé en argument
5     (effet de bord).
6     """
7     bulletin.append(note)
8
9     notes_ali = [12, 15]
10    ajouter_note(notes_ali, 18)
11    print(notes_ali) # [12, 15, 18]
```

Quand une fonction
modifie un argument
mutable, on
de bord
voul
documen
dar

POUR ALLER PLUS LOIN

L'opérateur `is` teste si deux variables désignent le *même objet* (même adresse mémoire), tandis que `==` teste l'égalité des *valeurs*. Après `b = a`, on a `b is a` qui vaut `True`. Après `b = list(a)`, on a `b is a` qui vaut `False` mais `b == a` qui vaut `True`.

» **Python À la machine.** Crée une liste `original = [1, 2, 3]`. Fais un alias `alias = original` et une copie `copie = list(original)`. Modifie `alias[0] = 99`. Affiche les trois variables et vérifie : `original` est modifié, `copie` ne l'est pas. Teste `alias is original` et `copie is original`.

MÉTHODE M1 — CHOISIR ENTRE TUPLE ET LISTE

Quand l'utiliser ? Quand tu dois stocker plusieurs valeurs ensemble et que tu hésites entre un tuple et une liste.

Pas à pas :

1. Demande-toi si les données seront modifiées après création (ajout, suppression, remplacement).
2. Si **non** (données fixes) : utilise un **tuple**. Exemples : coordonnées d'un point, date de naissance, résultat renvoyé par une fonction.
3. Si **oui** (données évolutives) : utilise une **liste**. Exemples : liste de scores en cours de partie, panier d'achats.
4. En cas de doute, pose-toi la question : « cette collection va-t-elle grandir ou changer ? »

Exemple :

```
1 # Données fixes : on choisit un tuple
2 position = (3, 7)
3 couleur_rgb = (255, 128, 0)
4
5 # Données évolutives : on choisit une liste
6 scores = [12, 8, 15]
```

```
7 scores.append(20) # on ajoute un score
```

Pièges :

- ▶ Utiliser une liste pour des données qui ne changent jamais : cela fonctionne, mais un tuple exprime mieux l'intention et protège contre les modifications accidentelles.
- ▶ Tenter de modifier un tuple avec `t[0] = 5` provoque une `TypeError`.

Vérifier : ton choix est cohérent si tu n'as jamais besoin de `append`, `remove` ou d'affectation par indice sur un tuple.

S'entraîner : exercices C1.

MÉTHODE M2 — CONSTRUIRE UN TABLEAU EN COMPRÉHENSION

Quand l'utiliser ? Quand tu veux créer une liste à partir d'une règle de calcul ou d'un filtre, en une seule ligne.

Pas à pas :

1. Identifie la **source** : sur quoi itères-tu ? (un range, une liste existante, etc.)
2. Identifie l'**expression** : que calcules-tu pour chaque élément ?
3. (Optionnel) Identifie la **condition** : quels éléments gardes-tu ?
4. Écris la compréhension selon le patron : `[expr for var in source if cond]`.

Exemple :

```
1 # Carres des entiers de 0 à 9
2 carres = [x**2 for x in range(10)]
3 # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
4
5 # Garder uniquement les nombres pairs d'une liste
6 nombres = [3, 8, 12, 5, 20, 7]
7 pairs = [n for n in nombres if n % 2 == 0]
8 # [8, 12, 20]
```

Pièges :

- ▶ Oublier les crochets `[...]` : sans eux, tu obtiens un générateur, pas une liste.
- ▶ Placer le `if` avant le `for` : la condition de filtrage se place *après* le `for`.
- ▶ Écrire une compréhension trop complexe : si tu as besoin de plus de deux lignes de logique, préfère une boucle classique.

Vérifier : affiche la liste obtenue avec `print` et contrôle sa longueur avec `len`.

S'entraîner : exercices C2.

MÉTHODE M3 — PARCOURIR UNE GRILLE LIGNE PAR LIGNE

Quand l'utiliser ? Quand tu travailles avec un tableau de tableaux (grille, image, plateau de jeu) et que tu dois accéder à chaque case.

Pas à pas :

1. Repère le nombre de lignes : `nb_lignes = len(grille)`.
2. Repère le nombre de colonnes : `nb_colonnes = len(grille[0])`.
3. Écris une première boucle `for i in range(nb_lignes)` pour parcourir les lignes.
4. Écris une seconde boucle imbriquée `for j in range(nb_colonnes)` pour parcourir les colonnes.
5. Accède à la case courante avec `grille[i][j]`.

Exemple :

```

1 grille = [
2     [1, 2, 3],
3     [4, 5, 6],
4 ]
5
6 for i in range(len(grille)):
7     for j in range(len(grille[0])):
8         print(grille[i][j], end=" ")
9     print() # retour a la ligne apres chaque ligne
10 # Affiche :
11 # 1 2 3
12 # 4 5 6

```

Pièges :

- ▶ Confondre lignes et colonnes : grille[i] est la ligne i ; grille[i][j] est la case à la ligne i, colonne j.
- ▶ Inverser les indices : grille[j][i] au lieu de grille[i][j] donne un résultat faux (et parfois une erreur d'indice).
- ▶ Supposer que toutes les lignes ont la même longueur sans le vérifier.

Vérifier : affiche i, j, grille[i][j] dans la boucle pour contrôler que chaque case est bien visitée.

S'entraîner : exercices C3.

MÉTHODE M4 — CHERCHER DANS UN DICTIONNAIRE SANS ERREUR

Quand l'utiliser ? Quand tu veux accéder à une valeur dans un dictionnaire sans risquer une `KeyError` si la clé est absente.

Pas à pas :

1. **Méthode 1 — tester avec in :** vérifie si la clé existe avant d'y accéder.
2. **Méthode 2 — utiliser .get() :** appelle dico.get(cle, valeur_par_defaut) qui renvoie la valeur par défaut si la clé est absente (au lieu de provoquer une erreur).
3. Choisis la méthode adaptée : in quand tu dois exécuter un bloc particulier selon la présence de la clé ; .get() quand tu veux simplement récupérer une valeur avec un repli.

Exemple :

```

1 inventaire = {"pommes": 5, "bananes": 3}
2
3 # Methode 1 : tester avec in
4 if "cerises" in inventaire:
5     print(inventaire["cerises"])
6 else:
7     print("Pas de cerises en stock")
8
9 # Methode 2 : utiliser .get()
10 qte = inventaire.get("cerises", 0)
11 print(qte) # 0 (valeur par default)

```

Pièges :

- ▶ Accéder directement avec dico[cle] sans vérification : si la clé est absente, Python lève une `KeyError`.
- ▶ Oublier le second argument de .get() : sans valeur par défaut, .get() renvoie `None` (pas une erreur, mais potentiellement inattendu).
- ▶ Confondre in sur un dictionnaire (cherche parmi les clés) et in sur une liste (cherche parmi les éléments).

Vérifier : teste ton code avec une clé présente *et* une clé absente pour confirmer qu'aucune erreur ne se produit.

S'entraîner : exercices C4.

MÉTHODE M5 — COPIER UNE LISTE SANS ALIAS

Quand l'utiliser ? Quand tu veux créer une copie indépendante d'une liste, de sorte que modifier la copie ne modifie pas l'original.

Pas à pas :

1. Identifie le problème : copie = originale ne crée **pas** une copie, mais un **alias** (les deux noms désignent le même objet en mémoire).
2. Pour obtenir une vraie copie, utilise l'une de ces méthodes :
 - ▶ copie = list(original)
 - ▶ copie = originale[:] (tranche complète)
3. Vérifie que les deux listes sont bien distinctes en modifiant la copie et en affichant l'original.

Exemple :

```

1 originale = [1, 2, 3]
2
3 # Alias : les deux variables pointent vers le meme objet
4 alias = originale
5 alias.append(4)
6 print(original) # [1, 2, 3, 4] -- modifiée aussi !
7
8 # Copie indépendante
9 originale = [1, 2, 3]
10 copie = list(original)
11 copie.append(4)
12 print(original) # [1, 2, 3] -- inchangée
13 print(copie)    # [1, 2, 3, 4]
```

Pièges :

- ▶ Croire que = copie une liste : c'est l'erreur la plus fréquente. L'affectation crée un alias, pas une copie.
- ▶ Attention aux listes imbriquées : list() et [:] ne font qu'une copie *superficielle*. Si la liste contient d'autres listes, les sous-listes restent partagées.

Vérifier : après la copie, teste copie is originale. Si le résultat est False, tu as bien deux objets distincts.

S'entraîner : exercices C5.

Exercice 1 ♦

⌚ 8 min

On considère le programme suivant :

```

1 notes = [12, 15, 9]
2 copie = notes
3 copie.append(18)
4 print(notes)
5 print(len(copie))
```

1. Donner, sans exécuter le programme, ce qu'affichent les deux instructions print.
2. Expliquer pourquoi la liste notes est modifiée alors que l'on n'a appelé append que sur copie.
3. Modifier la ligne 2 pour que notes ne soit pas modifiée.

Exercice 2

5 min

On donne le tuple suivant :

```
1 station = ("Tunis", 36.8, 10.2)
```

1. Donner le type de station et le nombre de ses éléments.
2. Donner la valeur de station[0] et de station[2].
3. Peut-on écrire station[1] = 37.0 ? Justifier.

8 min

Exercice 3

Écrire une fonction distance(a, b) qui prend deux tuples de coordonnées (x, y) et renvoie la distance euclidienne entre les deux points.

Exemples :

```
>> distance((0, 0), (3, 4))
5.0
>> distance((1, 1), (1, 1))
0.0
```

5 min

Exercice 4

On donne le tuple suivant, qui représente un joueur d'e-sport :

```
1 joueur = ("Fatou", 2350, "diamant")
2 nom, elo, rang = joueur
3 print(nom)
4 print(elo > 2000)
5 print(joueur[2])
```

1. Donner, sans exécuter le programme, ce qu'affichent les trois instructions print.
2. Quel mécanisme Python permet d'écrire nom, elo, rang = joueur ?
3. Peut-on écrire joueur[1] = 2400 ? Justifier.

5 min

Exercice 5

On donne la liste de tuples suivante, contenant des relevés de température :

```
1 releves = [("Paris", 32.5), ("Lyon", 35.1), ("Lille", 28.7)]
2 for ville, temp in releves:
3     if temp > 30:
4         print(ville)
```

1. Donner, sans exécuter le programme, ce qu'affiche ce code.
2. Quel est le type de chaque élément de la liste releves ?
3. Donner la valeur de releves[1][0].

8 min

Exercice 6

Écrire une fonction coordonnees_milieu(a, b) qui prend deux tuples représentant des coordonnées (x, y) et renvoie le tuple des coordonnées du milieu du segment [AB].

Exemples :


```

>> coordonnees_milieu((0, 0), (4, 6))
(2.0, 3.0)
>> coordonnees_milieu((-2, 3), (2, -3))
(0.0, 0.0)

```

Exercice 7

5 min

On modélise un match d'e-sport par un tuple (equipe1, equipe2, score1, score2).

```

1 match = ("TeamA", "TeamB", 2, 1)
2 equipe1, equipe2, score1, score2 = match
3 resultat = equipe1 if score1 > score2 else equipe2
4 print(resultat)
5 print(score1 + score2)

```

1. Donner, sans exécuter le programme, ce qu'affichent les deux print.
2. Que contient la variable resultat si le match est ("X", "Y", 0, 3) ?

Exercice 8

10 min

On représente une station météo par un tuple (nom, temperature).

Écrire une fonction station_la_plus_chaude(stations) qui prend une liste non vide de tuples et renvoie le nom de la station ayant la température la plus élevée.

Exemple :

```

>> station_la_plus_chaude([("Paris", 32), ("Lyon", 35), ("Lille", 28)])
'Lyon'

```

Exercice 9

10 min

On représente chaque joueur d'e-sport par un tuple (nom, elo, rang).

1. Écrire une fonction filtrer_par_rang(joueurs, rang_min) qui prend une liste de tuples et un entier rang_min, et renvoie la liste des tuples dont le score Elo est supérieur ou égal à rang_min.
2. Tester ta fonction avec la liste suivante :

```

1 joueurs = [("Alice", 2400, "diamant"),
2             ("Bob", 1800, "or"),
3             ("Clara", 2100, "platine")]

```

Exercice 10

8 min

On considère le programme suivant :

```

1 def swap(t):
2     return (t[1], t[0])
3
4 a = (3, 7)
5 b = swap(a)
6 print(a)
7 print(b)
8 c = (1, 2, 3)
9 print(swap(c))

```

1. Donner, sans exécuter, les trois affichages produits.
2. Le tuple a est-il modifié par l'appel `swap(a)` ? Justifier.
3. Que se passe-t-il si on appelle `swap(c)` avec `c = (1, 2, 3)` ? Le troisième élément est-il perdu ?

⌚ 12 min

Exercice 11 ◆◆

On modélise les résultats d'un tournoi par une liste de tuples (`equipe`, `score_pour`, `score_contre`).

Écrire une fonction `classer_matches(matches)` qui renvoie un tuple de trois listes : les équipes victorieuses, les équipes perdantes et les matchs nuls.

Exemple :

```
>>> classer_matches([("A", 3, 1), ("B", 0, 2), ("C", 1, 1)])
(['A'], ['B'], ['C'])
```

⌚ 15 min

Exercice 12 ◆◆◆

On dispose d'une liste de tuples (`x`, `y`) représentant les positions de stations météo sur une carte.

Écrire une fonction `enveloppe_convexe_1d(points)` qui renvoie un tuple de deux tuples : le coin inférieur gauche (`x_min`, `y_min`) et le coin supérieur droit (`x_max`, `y_max`) du rectangle englobant toutes les stations.

Exemple :

```
>>> enveloppe_convexe_1d([(1, 2), (3, 0), (-1, 5)])
((-1, 0), (3, 5))
```

Indication : parcourir tous les points pour trouver les extremums.

⌚ 15 min

Exercice 13 ◆◆◆

On représente un classement e-sport par une liste de tuples (`nom`, `elo`).

Écrire une fonction `top_joueurs(joueurs, n)` qui renvoie un tuple contenant les noms des `n` meilleurs joueurs, classés par score Elo décroissant.

Tu peux utiliser un tri par sélection ou tout autre algorithme de tri vu en cours.

Exemple :

```
>>> joueurs = [("Alice", 2400), ("Bob", 1800),
...           ("Clara", 2600), ("Dan", 2100)]
>>> top_joueurs(joueurs, 2)
('Clara', 'Alice')
```

Exercice 14 ◆

⌚ 5 min

On donne le tableau suivant :

```
1 temperatures = [15, 22, 18, 30, 25]
2 print(temperatures[0])
3 print(temperatures[-1])
4 print(len(temperatures))
5 temperatures[2] = 20
6 print(temperatures)
```

1. Donner, sans exécuter le programme, ce qu'affichent les quatre `print`.
2. Pourquoi peut-on écrire `temperatures[2] = 20` alors qu'on ne pourrait pas le faire avec un tuple ?

Exercice 15

5 min

On considère le programme suivant :

```
1 scores = [12, 8, 15, 3, 20]
2 resultat = []
3 for s in scores:
4     if s ≥ 10:
5         resultat.append(s)
6 print(resultat)
7 print(len(resultat))
```

1. Donner, sans exécuter le programme, les deux affichages produits.
2. Quel est le rôle de ce programme ?

Exercice 16

8 min

Écrire une fonction `moyenne(tab)` qui prend un tableau non vide d'entiers et renvoie la moyenne de ses éléments.

Exemples :

```
>>> moyenne([10, 20, 30])
20.0
>>> moyenne([12, 8, 15, 3, 20])
11.6
```

Exercice 17

8 min

On considère le programme suivant :

```
1 notes = [12, 8, 15, 6, 14, 9]
2 tab = [n for n in notes if n ≥ 10]
3 print(tab)
4 doubles = [n * 2 for n in [1, 2, 3]]
5 print(doubles)
```

1. Donner, sans exécuter, les deux affichages.
2. Réécrire la construction de `tab` en utilisant une boucle `for` et `append` (sans compréhension de liste).

Exercice 18

10 min

Écrire une fonction `indice_maximum(tab)` qui prend un tableau non vide d'entiers et renvoie l'indice de son plus grand élément. En cas d'égalité, on renvoie le plus petit indice.

Exemples :

```
>>> indice_maximum([3, 7, 2, 9, 1])
3
>>> indice_maximum([5, 5, 5])
0
```

Exercice 19

10 min

On considère la fonction suivante :

```

1 def mystere(tab):
2     resultat = []
3     for i in range(len(tab)):
4         if tab[i] not in resultat:
5             resultat.append(tab[i])
6     return resultat
7 print(mystere([3, 1, 3, 2, 1, 3]))

```

1. Donner, sans exécuter, l’affichage produit.
2. Décrire en une phrase le rôle de cette fonction.
3. Réécrire le corps de la fonction en parcourant directement les éléments (sans utiliser les indices).

12 min

Exercice 20

Une station météo enregistre des températures horaires dans un tableau. On veut détecter les valeurs anormales.

Écrire une fonction `temperatures_anormales(relevs, seuil_bas, seuil_haut)` qui renvoie la liste des tuples (indice, valeur) pour chaque relevé en dehors de l’intervalle `[seuil_bas, seuil_haut]`.

Exemple :

```

>> temperatures_anormales([15, 42, 18, -3, 22], 0, 40)
[(1, 42), (3, -3)]

```

10 min

Exercice 21

On considère la fonction suivante :

```

1 def decaler(tab):
2     dernier = tab[-1]
3     for i in range(len(tab) - 1, 0, -1):
4         tab[i] = tab[i - 1]
5     tab[0] = dernier
6
7 scores = [10, 20, 30, 40]
8 decaler(scores)
9 print(scores)

```

1. Donner, sans exécuter, l’affichage produit. Détailler l’état du tableau après chaque itération de la boucle.
2. Décrire en une phrase le rôle de cette fonction.
3. La fonction renvoie-t-elle une valeur ? Comment le tableau est-il modifié ?

8 min

Exercice 22

Compléter la trace du programme suivant, qui affiche les élèves ayant la moyenne :

```

1 eleves = ["Alice", "Bob", "Clara"]
2 notes = [14, 8, 17]
3 for i in range(len(eleves)):
4     if notes[i] ≥ 10:
5         print(eleves[i], ":", notes[i])

```

1. Donner les affichages produits.
2. Quel est le lien entre les deux tableaux `eleves` et `notes` ?

3. Pourquoi utilise-t-on `range(len(elevés))` plutôt que `for e in élèves` ?

15 min

Exercice 23

On dispose de deux tableaux d'entiers, chacun trié par ordre croissant.

Écrire une fonction `fusionner(tab1, tab2)` qui renvoie un nouveau tableau contenant tous les éléments des deux tableaux, trié par ordre croissant, sans utiliser de fonction de tri.

Exemple :

```
>>> fusionner([1, 3, 5], [2, 4, 6])
[1, 2, 3, 4, 5, 6]
```

Indication : utiliser deux indices parcourant chacun des tableaux.

Exercice 24

15 min

En météorologie, on lisse les relevés de température en calculant des **moyennes glissantes** : la valeur en position i est remplacée par la moyenne des k valeurs consécutives commençant en i .

Écrire une fonction `moyennes_glissantes(relevés, k)` qui renvoie le tableau des moyennes glissantes de taille k . Ce tableau contient $n - k + 1$ éléments (où n est la longueur du tableau).

Exemple :

```
>>> moyennes_glissantes([10, 20, 30, 40, 50], 3)
[20.0, 30.0, 40.0]
```

Exercice 25

5 min

On donne la grille suivante :

```
1 grille = [[1, 2, 3],
2           [4, 5, 6]]
3 print(grille[0][1])
4 print(grille[1][2])
5 print(len(grille))
6 print(len(grille[0]))
```

1. Donner, sans exécuter, les quatre affichages.
2. Combien de lignes et de colonnes possède cette grille ?
3. Donner l'expression permettant d'accéder à la valeur 4.

Exercice 26

8 min

Une grille représente les températures relevées en différents points d'une carte :

```
1 carte = [[20, 22, 19],
2           [25, 28, 23],
3           [18, 17, 21]]
4 total = 0
5 for ligne in carte:
6     for val in ligne:
7         total = total + val
8 print(total)
```

1. Donner, sans exécuter, l'affichage produit.

2. Modifier le code pour qu'il affiche la moyenne des températures de la grille.

⌚ 8 min

Exercice 27 ♦

Écrire une fonction `creer_grille(n, p, valeur)` qui renvoie une grille de `n` lignes et `p` colonnes, où chaque case contient `valeur`.

Exemple :

```
>> creer_grille(2, 3, 0)
[[0, 0, 0], [0, 0, 0]]
```

Attention : ne pas écrire `[[valeur] * p] * n`. Pourquoi ?

⌚ 8 min

Exercice 28 ♦

On considère le programme suivant :

```
1 grille = [[0, 0, 0],
2           [0, 0, 0],
3           [0, 0, 0]]
4 for i in range(3):
5     grille[i][i] = 1
6 for ligne in grille:
7     print(ligne)
```

1. Donner, sans exécuter, les trois affichages produits.
2. Quel objet mathématique cette grille représente-t-elle ?

⌚ 10 min

Exercice 29 ♦♦

On représente une carte de températures par une grille (liste de listes).

Écrire une fonction `maximum_grille(grille)` qui renvoie un tuple (ligne, colonne, valeur) correspondant à la position et la valeur du maximum dans la grille.

Exemple :

```
>> maximum_grille([[1, 5], [9, 3]])
(1, 0, 9)
```

⌚ 12 min

Exercice 30 ♦♦

Écrire une fonction `transposer(grille)` qui renvoie la transposée d'une grille : les lignes deviennent les colonnes et réciproquement.

Exemple :

```
>> transposer([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
```

La grille d'entrée ne doit pas être modifiée.

⌚ 12 min

Exercice 31 ♦♦

Dans une grille binaire (contenant des 0 et des 1), on veut compter les voisins d'une case.

Écrire une fonction `compter_voisins(grille, lig, col)` qui renvoie le nombre de cases voisines (horizontales, verticales et diagonales) contenant la valeur 1, sans compter la case elle-même.

Exemple :

```

>> g = [[0, 1, 0], [1, 1, 0], [0, 0, 1]]
>> compter_voisins(g, 1, 1)
3
>> compter_voisins(g, 0, 0)
3

```

Attention : gérer les bords de la grille.

Exercice 32 ♦♦

⌚ 10 min

Trois équipes d'e-sport ont chacune disputé trois matches. Leurs scores sont enregistrés dans la grille suivante :

```

1 scores = [[10, 8, 12],
2           [15, 11, 9],
3           [7, 14, 13]]
4 moyennes = []
5 for ligne in scores:
6     s = 0
7     for v in ligne:
8         s = s + v
9     moyennes.append(s / len(ligne))
10 print(moyennes)

```

1. Donner, sans exécuter, l'affichage produit.
2. Modifier le code pour n'afficher que la moyenne la plus élevée.

Exercice 33 ♦♦♦

⌚ 15 min

On dispose d'une carte de températures sous forme de grille. On veut créer une **carte binaire** identifiant les zones de chaleur.

1. Écrire une fonction `zone_chaude(carte, seuil)` qui renvoie une nouvelle grille de même taille contenant 1 si la température est supérieure ou égale au seuil, et 0 sinon.
2. Tester avec la carte :

```

1 carte = [[30, 25, 32],
2          [28, 35, 31],
3          [22, 19, 27]]

```

et un seuil de 30.

3. Proposer une fonction qui compte le nombre de cases à 1 dans la grille résultat.

Exercice 34 ♦♦♦

⌚ 20 min

On souhaite effectuer une rotation de 90 degrés dans le sens horaire d'une grille.

1. Sur papier, dessiner la grille `[[1, 2], [3, 4]]` puis sa version après rotation de 90 degrés.
2. Écrire une fonction `rotation_90(grille)` qui renvoie une nouvelle grille correspondant à la rotation. Si la grille d'entrée a `n` lignes et `p` colonnes, la grille résultat a `p` lignes et `n` colonnes.
3. Vérifier qu'appliquer quatre fois la rotation redonne la grille initiale.

Exemple :

```

>> rotation_90([[1, 2], [3, 4]])
[[3, 1], [4, 2]]

```

Exercice 35

5 min

On donne le dictionnaire suivant :

```
1 eleve = {"nom": "Alice", "classe": "1NSI", "moyenne": 14.5}
2 print(eleve["nom"])
3 print(eleve["moyenne"])
4 print(len(eleve))
```

1. Donner, sans exécuter, les trois affichages.
2. Quelles sont les clés de ce dictionnaire ? Quelles sont les valeurs ?
3. Écrire l'instruction qui ajoute la clé "email" avec la valeur "alice@lycee.fr".

5 min

Exercice 36

On considère le programme suivant :

```
1 meteo = {"ville": "Lyon", "temp": 28, "vent": 15}
2 meteo["temp"] = 30
3 meteo["humidite"] = 65
4 print(meteo["temp"])
5 print("humidite" in meteo)
6 print("pluie" in meteo)
```

1. Donner, sans exécuter, les trois affichages.
2. Quelle est la différence entre modifier une clé existante et ajouter une nouvelle clé ?

8 min

Exercice 37

Écrire une fonction `creer_joueur(nom, elo, rang)` qui renvoie un dictionnaire avec les clés "nom", "elo" et "rang".

Exemple :

```
>> creer_joueur("Alice", 2400, "diamant")
{'nom': 'Alice', 'elo': 2400, 'rang': 'diamant'}
```

8 min

Exercice 38

On considère le programme suivant :

```
1 station = {"nom": "Paris", "temp": 25, "vent": 10}
2 for cle in station:
3     print(cle, ":", station[cle])
```

1. Donner, sans exécuter, les affichages produits.
2. Que parcourt la boucle `for cle in station` : les clés, les valeurs, ou les deux ?
3. Réécrire la boucle en utilisant `station.items()`.

10 min

Exercice 39

Écrire une fonction `compter_occurrences(tab)` qui prend un tableau et renvoie un dictionnaire associant à chaque élément son nombre d'apparitions.

Exemple :


```
>> compter_occurrences(["a", "b", "a", "c", "b", "a"])
{'a': 3, 'b': 2, 'c': 1}
```

Exercice 40

10 min

On considère la fonction suivante :

```
1 def fusionner_dicos(d1, d2):
2     resultat = {}
3     for cle in d1:
4         resultat[cle] = d1[cle]
5     for cle in d2:
6         resultat[cle] = d2[cle]
7     return resultat
8
9 a = {"x": 1, "y": 2}
10 b = {"y": 10, "z": 3}
11 print(fusionner_dicos(a, b))
```

1. Donner, sans exécuter, l’affichage produit.
2. Que se passe-t-il lorsqu’une clé est présente dans les deux dictionnaires ?
3. Les dictionnaires a et b sont-ils modifiés ?

Exercice 41

12 min

On dispose d’une liste de dictionnaires représentant des élèves, chacun ayant les clés “nom”, “classe” et “note”.

Écrire une fonction `moyenne_par_classe(elevs)` qui renvoie un dictionnaire associant à chaque classe la moyenne des notes de ses élèves.

Exemple :

```
>> elevs = [{"nom": "A", "classe": "1A", "note": 12},
...         {"nom": "B", "classe": "1B", "note": 15},
...         {"nom": "C", "classe": "1A", "note": 14}]
>> moyenne_par_classe(elevs)
{'1A': 13.0, '1B': 15.0}
```

Exercice 42

8 min

On considère le code suivant :

```
1 joueur = {"nom": "Bob", "elo": 1800}
2 print(joueur["rang"])
```

1. Quelle erreur ce programme produit-il ? Pourquoi ?
2. Proposer deux façons différentes d’éviter cette erreur :
 - ▶ en testant l’existence de la clé avant l’accès ;
 - ▶ en utilisant la méthode `get`.
3. Réécrire le code pour qu’il affiche “inconnu” si la clé “rang” n’existe pas.

Exercice 43

12 min

Écrire une fonction `inverser_dico(d)` qui prend un dictionnaire et renvoie un nouveau dictionnaire où les clés et les valeurs sont échangées.

On suppose que toutes les valeurs du dictionnaire d'entrée sont distinctes et peuvent servir de clés.

Exemple :

```
>> inverser_dico({"a": 1, "b": 2, "c": 3})
{1: 'a', 2: 'b', 3: 'c'}
```

15 min

Exercice 44

On modélise un tournoi d'e-sport par une liste de tuples (equipe1, score1, equipe2, score2).

Écrire une fonction `classement_tournoi(matches)` qui renvoie un dictionnaire associant à chaque équipe son total de points : 3 points pour une victoire, 1 point pour un match nul, 0 pour une défaite.

Exemple :

```
>> matches = [("A", 2, "B", 1), ("A", 0, "C", 0), ("B", 3, "C", 1)]
>> classement_tournoi(matches)
{'A': 4, 'B': 3, 'C': 1}
```

20 min

Exercice 45

On dispose d'une liste de dictionnaires représentant des stations météo, chacun ayant les clés "nom", "region" et "temp".

1. Écrire une fonction `stations_par_region(stations)` qui renvoie un dictionnaire dont les clés sont les régions et les valeurs sont les listes des noms de stations dans chaque région.
2. Compléter par une fonction `region_la_plus_chaude(stations)` qui renvoie le nom de la région ayant la température moyenne la plus élevée.

Exemple :

```
>> stations = [{"nom": "Lyon", "region": "ARA", "temp": 30},
...             {"nom": "Paris", "region": "IDF", "temp": 28},
...             {"nom": "Grenoble", "region": "ARA", "temp": 27}]
>> stations_par_region(stations)
{'ARA': ['Lyon', 'Grenoble'], 'IDF': ['Paris']}
```

5 min

Exercice 46

On considère le programme suivant :

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a)
5 print(a is b)
```

1. Donner, sans exécuter, les deux affichages.
2. Pourquoi la liste a est-elle modifiée alors qu'on n'a appelé `append` que sur b ?
3. Que signifie `a is b` ?

Exercice 47

5 min

On considère le programme suivant :

```
1 a = [1, 2, 3]
```

```

2 b = list(a)
3 b.append(4)
4 print(a)
5 print(b)
6 print(a is b)

```

1. Donner, sans exécuter, les trois affichages.
2. Quelle est la différence avec `b = a` ?
3. Citer deux autres façons de créer une copie superficielle d'une liste.

Exercice 48

8 min

On considère le programme suivant :

```

1 def ajouter(tab, val):
2     tab.append(val)
3
4 scores = [10, 20]
5 ajouter(scores, 30)
6 print(scores)

```

1. Donner, sans exécuter, l'affichage produit.
2. La fonction `ajouter` ne contient pas de `return`. Comment se fait-il que la liste `scores` soit modifiée ?
3. On parle d'**effet de bord**. Expliquer ce terme.

Exercice 49

On considère le programme suivant :

```

1 t = (1, 2, 3)
2 t[0] = 10
3 print(t)

```

1. Que se passe-t-il lorsqu'on exécute ce programme ? Quelle erreur obtient-on ?
2. Expliquer pourquoi on dit qu'un tuple est **immuable** (non mutable).
3. Donner un avantage d'utiliser un tuple plutôt qu'une liste quand on ne souhaite pas modifier les données.

Exercice 50

On considère le programme suivant :

```

1 grille = [[0, 0]] * 3
2 grille[0][0] = 1
3 for ligne in grille:
4     print(ligne)

```

1. Donner, sans exécuter, les trois affichages. Le résultat est-il celui attendu ?
2. Expliquer pourquoi toutes les lignes sont modifiées alors qu'on n'a changé que `grille[0][0]`.
3. Proposer une construction correcte de la grille qui évite ce piège.

Exercice 51

Un élève écrit le code suivant pour supprimer les valeurs négatives d'un tableau :

```
1 def supprimer_negatifs_v1(tab):
2     for val in tab:
3         if val < 0:
4             tab.remove(val)
5     return tab
```

1. Tester mentalement `supprimer_negatifs_v1([5, -2, -3, 8])`. Le résultat est-il correct ? Expliquer le problème.
2. Voici une version corrigée :

```
1 def supprimer_negatifs(tab):
2     resultat = []
3     for val in tab:
4         if val ≥ 0:
5             resultat.append(val)
6     return resultat
```

Pourquoi cette version est-elle plus fiable ? Le tableau d'origine est-il modifié ?

Exercice 52



⌚ 10 min

On considère le programme suivant :

```
1 equipe = {"nom": "Phoenix", "score": 100}
2 copie = equipe
3 copie["score"] = 200
4 print(equipe["score"])
5 print(equipe is copie)
```

1. Donner, sans exécuter, les deux affichages.
2. Les dictionnaires sont-ils mutables ? Justifier.
3. Réécrire le code pour que la modification de copie ne modifie pas equipe. On pourra utiliser `dict(equipe)`.

Exercice 53



⌚ 12 min

On considère le programme suivant :

```
1 grille = [[1, 2], [3, 4]]
2 copie = list(grille)
3 copie[0][0] = 99
4 print(grille[0][0])
5 copie[0] = [10, 20]
6 print(grille[0])
```

1. Donner, sans exécuter, les deux affichages.
2. Expliquer pourquoi `grille[0][0]` vaut 99 alors qu'on a modifié copie.
3. On parle de **copie superficielle**. Qu'est-ce que cela signifie ?
4. Proposer une façon de créer une copie profonde de la grille (sans utiliser le module `copy`).

Exercice 54



⌚ 15 min

On veut créer une fonction qui copie entièrement une grille (liste de listes) sans qu'aucune modification de la copie n'affecte l'original.

1. Écrire une fonction `copie_profonde_grille(grille)` qui renvoie une copie profonde de la grille.
2. Vérifier que la modification de la copie ne modifie pas l'original :

```
1 g = [[1, 2], [3, 4]]
2 c = copie_profonde_grille(g)
3 c[0][0] = 99
4 print(g[0][0]) # doit afficher 1
```

3. Expliquer pourquoi `list(grille)` ne suffit pas.

Exercice 55



20 min

On représente des équipes d'e-sport par une liste de dictionnaires ayant les clés "nom" et "scores" (une liste d'entiers).

1. Écrire une fonction `historique_scores(equipes, resultats)` qui :
 - ▶ prend la liste des équipes et une liste de tuples (nom, score) ;
 - ▶ renvoie une **nouvelle** liste d'équipes où les scores ont été ajoutés ;
 - ▶ **ne modifie pas** la liste d'origine (ni les dictionnaires, ni les listes de scores qu'ils contiennent).
2. Tester avec :

```
1 equipes = [{"nom": "A", "scores": [10, 20]},
2             {"nom": "B", "scores": [15]}]
3 nouvelles = historique_scores(equipes, [("A", 30), ("B", 25)])
4 print(equipes[0]["scores"]) # [10, 20]
5 print(nouvelles[0]["scores"]) # [10, 20, 30]
```

3. Expliquer pourquoi il faut copier à la fois les dictionnaires et les listes de scores.

Coup de pouce 1. Demande-toi combien de listes existent réellement en mémoire après la ligne 2 : une ou deux ?

Coup de pouce 2. Dessine la mémoire : deux étiquettes (notes, copie) et des flèches vers les objets. Où pointent-elles ?

Coup de pouce 3. Compare `copie = notes` (partage de référence) et `copie = list(notes)` (nouvel objet). Rejoue le programme avec chacune.

Coup de pouce 1. Rappelle-toi qu'un tuple est entouré de parenthèses et qu'on accède à ses éléments par un indice commençant à 0.

Coup de pouce 2. Pour la question 3, essaie la commande dans la console Python et observe le message d'erreur.

Coup de pouce 1. Utilise la formule de distance : $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

Coup de pouce 2. En Python, la racine carrée se calcule avec `** 0.5`. Accède aux coordonnées du tuple avec `a[0]` et `a[1]`.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui utilise un tuple et le déballage (*unpacking*), puis d'expliquer pourquoi un tuple ne peut pas être modifié.

Coup de pouce 2. Commence par identifier ce que contient chaque variable après `nom, elo, rang = joueur` : chaque variable reçoit l'élément du tuple à la position correspondante.

Coup de pouce 1. On te demande de prévoir l'affichage d'un programme qui parcourt une liste de tuples et filtre selon une condition sur la température.

Coup de pouce 2. À chaque tour de boucle, `ville` et `temp` reçoivent les deux éléments du tuple courant (déballage). Vérifie pour chaque tuple si `temp > 30`.

Coup de pouce 1. On te demande d'écrire une fonction qui calcule les coordonnées du milieu d'un segment à partir de deux points représentés par des tuples (x, y).

Coup de pouce 2. La formule du milieu est $(\frac{x_A + x_B}{2}, \frac{y_A + y_B}{2})$. Accède aux coordonnées avec `a[0]`, `a[1]`, `b[0]`, `b[1]` et renvoie un tuple.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui utilise le déballage d'un tuple et une expression conditionnelle (... if ... else ...).

Coup de pouce 2. L'expression `equipe1 if score1 > score2 else equipe2` renvoie `equipe1` si la condition est vraie, sinon `equipe2`. Compare les scores pour déterminer `resultat`.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui accède aux éléments d'un tableau (liste) par leur indice, y compris un indice négatif, et qui modifie un élément.

Coup de pouce 2. Rappelle-toi que `temperatures[-1]` désigne le dernier élément du tableau. Pour la question 2, compare les listes (mutables) aux tuples (immuables).

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui construit un nouveau tableau en filtrant les éléments d'un tableau existant selon une condition.

Coup de pouce 2. Suis la boucle élément par élément : pour chaque valeur `s` de scores, vérifie si `s >= 10` ; si oui, elle est ajoutée à `resultat` avec `append`.

Coup de pouce 1. On te demande d'écrire une fonction qui calcule la moyenne des éléments d'un tableau d'entiers.

Coup de pouce 2. Parcourt le tableau avec une boucle `for` pour accumuler la somme dans une variable, puis divise cette somme par `len(tab)`.

Coup de pouce 1. On te demande de prévoir les affichages de deux constructions par compréhension de liste, puis de réécrire l'une d'elles avec une boucle `for` classique.

Coup de pouce 2. La syntaxe `[n for n in notes if n >= 10]` signifie : « pour chaque `n` dans `notes`, garde `n` seulement si `n >= 10` ». Pour réécrire, utilise un tableau vide, une boucle et `append`.

Coup de pouce 1. On te demande de tracer l'exécution d'un programme qui utilise deux tableaux parallèles (élèves et notes) et affiche ceux qui ont la moyenne.

Coup de pouce 2. L'indice `i` sert à accéder à la même position dans les deux tableaux : `eleves[i]` et `notes[i]` vont ensemble. C'est pour cela qu'on utilise `range(len(eleves))` et non `for e in eleves`.

Coup de pouce 1. On te demande de lire les valeurs d'une grille (tableau de tableaux) en utilisant la double indexation `grille[ligne][colonne]`.

Coup de pouce 2. `grille[0]` est la première ligne `[1, 2, 3]`, donc `grille[0][1]` est le deuxième élément de cette ligne. `len(grille)` donne le nombre de lignes.

Coup de pouce 1. On te demande de calculer la somme de toutes les valeurs d'une grille avec une double boucle, puis de modifier le code pour obtenir la moyenne.

Coup de pouce 2. La boucle externe parcourt chaque ligne, la boucle interne chaque valeur de la ligne. Pour la moyenne, divise le total par le nombre total de cases (lignes × colonnes).

Coup de pouce 1. On te demande d'écrire une fonction qui construit une grille de `n` lignes et `p` colonnes remplie d'une même valeur.

Coup de pouce 2. Utilise deux boucles imbriquées : la boucle externe crée chaque ligne, la boucle interne remplit la ligne avec `append`. Ajoute chaque ligne terminée à la grille.

Coup de pouce 3. `[[valeur] * p] * n` crée `n` références vers la *même* ligne : modifier une case les modifie toutes. C'est pourquoi il faut créer une nouvelle ligne à chaque itération.

Coup de pouce 1. On te demande de tracer l'exécution d'un programme qui modifie la diagonale d'une grille 3×3, puis d'identifier l'objet mathématique obtenu.

Coup de pouce 2. La boucle `for i in range(3)` place un 1 aux positions `grille[0][0]`, `grille[1][1]` et `grille[2][2]`, c'est-à-dire sur la diagonale.

Coup de pouce 1. On te demande de lire les valeurs d'un dictionnaire en utilisant ses clés, puis d'ajouter une nouvelle entrée.

Coup de pouce 2. Accède avec `eleve["nom"]`. Ajoute avec `eleve["email"] = "..."`. Compte avec `len(eleve)`.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui modifie une clé existante, ajoute une nouvelle clé et teste l'appartenance d'une clé à un dictionnaire.

Coup de pouce 2. La syntaxe est la même pour modifier et ajouter : `dico[cle] = valeur`. L'opérateur `in` teste si une *clé* (pas une valeur) est présente dans le dictionnaire.

Coup de pouce 1. On te demande d'écrire une fonction qui crée et renvoie un dictionnaire à partir de trois paramètres.

Coup de pouce 2. Construis le dictionnaire directement avec la syntaxe `{"nom": nom, "elo": elo, "rang": rang}` et renvoie-le avec `return`.

Coup de pouce 1. On te demande de prévoir les affichages d'un parcours de dictionnaire avec une boucle `for`, puis de réécrire ce parcours avec `.items()`.

Coup de pouce 2. `for cle in station` parcourt uniquement les *clés*. Pour obtenir la valeur associée, on écrit `station[cle]`. Avec `.items()`, la boucle devient `for cle, val in station.items()`.

Coup de pouce 1. On te demande de comprendre pourquoi modifier `b` modifie aussi `a` quand on écrit `b = a` avec une liste.

Coup de pouce 2. L'affectation `b = a` ne copie pas la liste : `a` et `b` sont deux noms pour le *même* objet en mémoire. `a is b` renvoie `True` car c'est la même référence.

Coup de pouce 1. On te demande de comprendre la différence entre `b = a` (alias) et `b = list(a)` (copie) d'une liste, puis de citer d'autres façons de copier.

Coup de pouce 2. `list(a)` crée un *nouvel* objet contenant les mêmes éléments : modifier `b` ne modifie plus `a`. Pense aussi à `a[:]` et `a.copy()`.

Coup de pouce 1. On te demande de comprendre pourquoi une fonction sans `return` peut quand même modifier une liste passée en argument. C'est la notion d'effet de bord.

Coup de pouce 2. Quand tu passes une liste à une fonction, la fonction reçoit une *référence* vers le même objet. Tout `append` ou modification d'élément à l'intérieur de la fonction agit sur l'original.

Coup de pouce 1. On te demande de comprendre pourquoi l'affectation `t[0] = 10` provoque une erreur quand `t` est un tuple, et d'expliquer l'intérêt de l'immuabilité.

Coup de pouce 2. Python lève une `TypeError` car un tuple est immuable : une fois créé, on ne peut ni ajouter, ni supprimer, ni modifier ses éléments. C'est une garantie que les données ne changeront pas.

TD 1 — Station météo connectée

Durée : 50 minutes. Objectif : manipuler tuples, tableaux et dictionnaires dans un contexte réaliste.

Une station météo enregistre des mesures toutes les heures. Chaque mesure est un tuple (heure, température, humidité). L'ensemble des mesures d'une journée est stocké dans un tableau.

```
1 mesures = [
2     (0, 15.2, 82), (1, 14.8, 85), (2, 14.1, 88),
3     (6, 13.5, 90), (7, 14.0, 87), (8, 15.5, 80),
4     (12, 22.3, 55), (13, 23.1, 52), (14, 23.8, 50),
5     (18, 20.1, 60), (19, 18.5, 65), (20, 17.2, 70),
6 ]
```

Partie A — Exploration (C1, C2)

Exercice 56

5 min

- Donner le type de `mesures[0]` et la valeur de `mesures[0][1]`.
- Écrire une fonction `temperature_max(mesures)` qui renvoie un tuple (heure, temp) correspondant à la température la plus élevée.

Exercice 57

8 min

Écrire une fonction `temperatures(mesures)` qui renvoie le tableau des températures extraites de chaque mesure, en compréhension de liste.

Partie B — Analyse (C4)

Exercice 58

10 min

On veut regrouper les mesures par tranche horaire dans un dictionnaire : "nuit" (0h–6h), "matin" (7h–12h), "après-midi" (13h–18h), "soir" (19h–23h).

Écrire une fonction `par_tranche(mesures)` qui renvoie un dictionnaire dont les clés sont les tranches et les valeurs sont les tableaux de mesures correspondantes.

Exercice 59

12 min

Écrire une fonction `resume(mesures)` qui renvoie un dictionnaire de la forme :

```
1 {"temp_min": 13.5, "temp_max": 23.8,
2  "hum_moy": 72.0, "nb_mesures": 12}
```

Partie C — Synthèse (C1, C2, C4)

Exercice 60

15 min

Écrire un programme complet qui :

1. Calcule le résumé de la journée.
2. Affiche les mesures de la tranche "après-midi".
3. Identifie l'heure la plus humide.

TD 2 — Classement d'un tournoi

Durée : 50 minutes. Objectif : croiser tableaux, dictionnaires et mutabilité dans un projet de classement.

Un tournoi oppose 6 joueurs sur 4 manches. Les résultats sont stockés dans une grille (tableau de tableaux) : chaque ligne est un joueur, chaque colonne une manche.

```
1 pseudos = ["Ace", "Bob", "Cat", "Dan", "Eve", "Fox"]
2 scores = [
3     [120, 150, 130, 140],
4     [180, 160, 170, 190],
5     [110, 140, 120, 100],
6     [200, 180, 210, 190],
7     [90, 110, 100, 120],
8     [160, 170, 150, 180],
9 ]
```

Partie A — Lecture de la grille (C3)

Exercice 61

5 min

1. Combien de lignes et de colonnes a la grille scores ?
2. Donner l'expression Python pour obtenir le score de Cat à la manche 3.
3. Écrire une fonction `total_joueur(scores, i)` qui renvoie la somme des scores du joueur d'indice `i`.

Partie B — Construction du classement (C2, C4)

Exercice 62

10 min

Écrire une fonction `classement(pseudos, scores)` qui renvoie un tableau de dictionnaires triés par total décroissant. Chaque dictionnaire contient les clés "pseudo", "total" et "rang".

Exemple de sortie (premier élément) :


```
1 {"pseudo": "Dan", "total": 780, "rang": 1}
```

Exercice 63 ♦♦

Écrire une fonction `meilleur_manche(scores, j)` qui renvoie l'indice du joueur ayant le meilleur score à la manche `j`.

Partie C — Mutabilité et copies (C5)

Exercice 64 ♦♦

On veut simuler un bonus : ajouter 10 points à chaque score de la manche 1, **sans modifier la grille originale**.

1. Un élève écrit :

```
1 copie = list(scores)
2 for i in range(len(copie)):
3     copie[i][0] += 10
```

Expliquer pourquoi la grille originale est aussi modifiée.

- Corriger le code pour que la grille originale reste intacte.
- Vérifier en affichant `scores[0][0]` avant et après.

Partie D — Synthèse (C2, C3, C4)

Exercice 65 ♦♦♦

Écrire un programme complet qui :

- Affiche le classement complet.
- Identifie le MVP (joueur avec la meilleure moyenne par manche).
- Crée un dictionnaire équipes regroupant les 6 joueurs en 2 équipes de 3 (indices 0–2 et 3–5), avec le total par équipe.

MINI-PROJET — CLASSEMENT D'UN TOURNOI DE E-SPORT

Un tournoi de e-sport oppose 8 joueurs sur 5 manches. Chaque joueur est représenté par un dictionnaire contenant son pseudo, son équipe et ses scores (un tableau d'entiers). L'objectif est de construire un programme complet qui gère le classement.

Jalon 1. Représenter les données. Définis un tableau `joueurs` contenant 8 dictionnaires avec les clés "pseudo", "equipe" et "scores" (tableau de 5 entiers). Vérifie que tu peux accéder au 3e score du 2e joueur.

Jalon 2. Calculer les totaux. Écris une fonction `total(joueur)` qui renvoie la somme des scores d'un joueur. Ajoute une clé "total" à chaque joueur.

Jalon 3. Trier le classement. Écris une fonction `classement(joueurs)` qui renvoie un nouveau tableau (pas un alias !) des joueurs triés par total décroissant. Indice : utilise `sorted` avec le paramètre `key`.

Jalon 4. Afficher les résultats. Écris une fonction `afficher_classement(joueurs)` qui affiche le classement sous la forme : 1. Ace (TeamA) – 850 pts.

Critères de validation (testables) : Jalon 1 : `accès joueurs[1]["scores"][2]` renvoie un entier ; Jalon 2 : `total(joueurs[0]) == somme des 5 scores` ; Jalon 3 : le premier du classement a le total le plus élevé ET joueurs n'est pas modifié (copie, pas alias) ; Jalon 4 : l'affichage montre rang, pseudo, équipe et total.

Extensions ♦♦♦ :

- ▶ Ajouter le classement par équipe (somme des totaux des joueurs de chaque équipe).
- ▶ Gérer les ex aequo (même total → départage par le meilleur score individuel).

- Exporter le classement dans un fichier texte.

Diagnostics du QCM — Types construits

Compare tes réponses et lis le diagnostic correspondant.

Q1. Réponse : B.

- Si A : tu confonds tuple (parenthèses) et liste (crochets) → M1.
- Si C : la virgule crée un tuple, pas un entier → D1.

Q2. Réponse : B.

- Si A : un tuple est non mutable, on ne peut pas modifier ses éléments → EF1.
- Si C : `t[0] = 5` n'ajoute pas, il tente une modification interdite → D1.

Q3. Réponse : C.

- Si A : le déballage associe le premier élément à `a` → D3.
- Si B : le déballage sépare les éléments, il ne copie pas le tuple → D3.

Q4. Réponse : B.

- Si A : `append` ajoute un élément, la longueur augmente de 1 → D5.
- Si C : un seul élément est ajouté par `append` → D5.

Q5. Réponse : A.

- Si B : cette expression produit 10 éléments (0 à 18), pas 5 → D6.
- Si C : aucun filtre pair, et 8 est exclu de `range(8)` → D6.

Q6. Réponse : B.

- Si A : l'indice 3 n'existe pas dans un tableau de 3 éléments (indices 0–2).
- Si C : Python ne renvoie pas `None` pour un indice invalide.

Q7. Réponse : B.

- Si A : tu confonds ligne et colonne. `g[1]` est `[3, 4]` → D7.
- Si C : `g[1]` n'est pas `[1, 2]` → D7.

Q8. Réponse : B.

- Si A : `*` crée des alias de la même ligne → D8.
- Si C : cette expression crée une grille 2×3, pas 3×2.

Q9. Réponse : B.

- Si A : chaque sous-liste est une ligne, il y en a 2 → D7.

Q10. Réponse : C.

- Si A : Python ne renvoie pas `None` mais lève une exception → EF3, D10.

Q11. Réponse : B.

- Si A : `append` est une méthode de liste, pas de dictionnaire → D10.

Q12. Réponse : B.

- Si A : cette boucle ne donne que les clés.
- Si C : cette boucle ne donne que les valeurs.

Q13. Réponse : B.

- Si A : `b = a` crée un alias, pas une copie → D11.

Q14. Réponse : B.

- Si A : ceci crée un alias, pas une copie → D11, D12.
- Si C : `t * 1` crée une copie superficielle, mais `list(t)` est plus explicite.

Q15. Réponse : B.

- Si A : la copie superficielle ne copie pas les sous-listes → EF4.

Q16. Réponse : B.

- Si A : tuple et str sont non mutables → D11.
- Si C : int est non mutable → D11.

QCM — Types construits

Pour chaque question, une seule réponse est correcte.

Q1. (C1) Que renvoie `type((3, 5))` ?

- A. `<class 'list'>`
- B. `<class 'tuple'>`
- C. `<class 'int'>`

Q2. (C1) Que se passe-t-il si on exécute ce code ?

```
1 t = (1, 2)
2 t[0] = 5
```

- A. t vaut (5, 2)
- B. Une erreur `TypeError`
- C. t vaut (1, 2, 5)

Q3. (C1) a, b = (10, 20). Quelle est la valeur de b ?

- A. 10
- B. (10, 20)
- C. 20

Q4. (C2) `t = [3, 1, 4]; t.append(1); print(len(t))`. Qu'affiche ce programme ?

- A. 3
- B. 4
- C. 5

Q5. (C2) Quelle expression construit la liste [0, 2, 4, 6, 8] ?

- A. `[x for x in range(10) if x % 2 == 0]`
- B. `[x * 2 for x in range(10)]`
- C. `[x for x in range(8)]`

Q6. (C2) `t = [10, 20, 30]; print(t[3])`. Qu'obtient-on ?

- A. 30
- B. `IndexError`
- C. None

Q7. (C3) `g = [[1, 2], [3, 4]]; print(g[1][0])`. Qu'affiche ce programme ?

- A. 2
- B. 3
- C. 1

Q8. (C3) Quelle expression crée une grille 3×2 de zéros sans alias ?

- A. `[[0, 0]] * 3`
- B. `[[0] * 2 for _ in range(3)]`
- C. `[[0 for _ in range(3)] for _ in range(2)]`

Q9. (C3) Combien de lignes et de colonnes a `g = [[5, 6, 7], [8, 9, 10]]` ?

- A. 3 lignes, 2 colonnes
- B. 2 lignes, 3 colonnes
- C. 6 lignes, 1 colonne

Q10. (C4) `d = {"a": 1, "b": 2}; print(d["c"])`. Qu'obtient-on ?

- A. None
- B. 0
- C. KeyError

Q11. (C4) Comment ajouter la clé "c" avec la valeur 3 à d ?

- A. `d.append("c", 3)`
- B. `d["c"] = 3`
- C. `d.add("c": 3)`

Q12. (C4) Quelle boucle parcourt clés ET valeurs d'un dictionnaire d ?

- A. `for k in d:`
- B. `for k, v in d.items():`
- C. `for v in d.values():`

Q13. (C5) `a = [1, 2]; b = a; b.append(3); print(a)`. Qu'affiche-t-il ?

- A. `[1, 2]`
- B. `[1, 2, 3]`
- C. Une erreur

Q14. (C5) Comment obtenir une copie indépendante d'une liste t ?

- A. `copie = t`
- B. `copie = list(t)`
- C. `copie = t * 1`

Q15. (C5) `g = [[1], [2]]; h = list(g); h[0].append(9); print(g[0])`.

- A. `[1]`
- B. `[1, 9]`
- C. Une erreur

Q16. (C5) Parmi ces types, lesquels sont mutables ?

- A. tuple et str
- B. list et dict
- C. int et list

3 Traitement de données en tables

3.1 Importer une table

DÉFINITION D1

Une **table** est une liste de dictionnaires ayant tous les mêmes clés. Chaque dictionnaire de la liste représente une **ligne** de la table ; chaque clé représente une **colonne**.

Exemple. Les adhérents d'un club sportif, sous forme de table :

```
1 adherents = [  
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},  
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},  
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},  
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},  
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},  
7 ]
```

DÉFINITION D2

Le format **CSV** (*comma-separated values*) représente une table dans un fichier texte : la première ligne contient les noms des colonnes (l'en-tête), chaque ligne suivante contient les valeurs séparées par une virgule (ou un autre séparateur, par exemple une tabulation pour un fichier « texte tabulé »).

Exemple. Le fichier adherents.csv correspondant à la table ci-dessus :

```
nom,age,categorie,cotisation  
Ali,16,junior,80  
Sami,15,junior,80  
Nour,17,junior,80  
Yasmine,22,senior,120  
Karim,19,senior,120
```

CODE DE RÉFÉRENCE — Importer un fichier CSV en table

```
1 import csv  
2  
3 def importer_csv(nom_fichier):  
4     """Importe un fichier CSV et renvoie une table (liste de dictionnaires).  
5  
6     Precondition : le fichier existe, la première ligne contient l'en-tete.  
7     """  
8     with open(nom_fichier, encoding="utf-8") as f:  
9         lecteur = csv.DictReader(f)  
10        return list(lecteur)  
11  
12 table = importer_csv("adherents.csv")
```

| Une *table* est un
de p-uplets nommés
(dictionnaires) qui
partagent les mêmes
descripteurs (les
clés) : chaque ligne
représente un
enregistrement, c'est-à-dire
une colonne un descripteur
commun à toutes les
lignes.

```
13 print(table[0]) # {'nom': 'Ali', 'age': '16', 'categorie': 'junior', 'cotisation': '80'}
```

ERREUR FRÉQUENTE

Oublier que les valeurs importées d'un CSV sont **toujours des chaînes de caractères** (str), même si elles « ressemblent » à des nombres. `table[0]["age"]` vaut la chaîne "16", pas l'entier 16 : il faut convertir explicitement avec `int(...)` ou `float(...)` avant de faire des calculs ou des comparaisons numériques.

CODE DE RÉFÉRENCE — Convertir les types après import

```
1 def convertir_types(table):
2     """Convertit les colonnes numeriques age et cotisation en entiers."""
3     for ligne in table:
4         ligne["age"] = int(ligne["age"])
5         ligne["cotisation"] = int(ligne["cotisation"])
6     return table
```

» **Python À la machine.** Crée un fichier `adherents.csv` avec le contenu de l'exemple ci-dessus, importe-le avec `importer_csv`, puis convertis les colonnes `age` et `cotisation` en entiers avec `convertir_types`. Vérifie avec `type(table[0]["age"])` que la conversion a bien eu lieu.

3.2 Recherche dans une table

Propriété Recherche par critère. Rechercher les lignes d'une table vérifiant un critère consiste à parcourir la table et à ne conserver que les lignes pour lesquelles une **expression booléenne** (le critère) est vraie. On peut combiner plusieurs conditions avec `and`, `or`, `not`.

CODE DE RÉFÉRENCE — Recherche par critère (compréhension de liste)

```
1 adherents = [
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 ]
8
9 def rechercher(table, critere):
10     """Renvoie les lignes de 'table' pour lesquelles critere(ligne) est vrai."""
11     return [ligne for ligne in table if critere(ligne)]
12
13 juniors_16plus = rechercher(adherents, lambda ligne: ligne["categorie"] == "junior"
14                             and ligne["age"] >= 16)
15 print([ligne["nom"] for ligne in juniors_16plus]) # ['Ali', 'Nour']
```

Propriété Recherche de doublons. Une table contient un **doublon** lorsque plusieurs lignes ont la même valeur pour une colonne censée être unique (par exemple un identifiant). On détecte les doublons en comptant les occurrences de chaque valeur de cette colonne.

CODE DE RÉFÉRENCE — Détecter les doublons d'une colonne

```
1 def doublons(table, colonne):
2     """Renvoie les valeurs de 'colonne' apparaissant plus d'une fois dans 'table'."""
3     valeurs = [ligne[colonne] for ligne in table]
4     return [v for v in set(valeurs) if valeurs.count(v) > 1]
5
6 adherents_avec_doublon = adherents + [{"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80}]
7 print(doublons(adherents_avec_doublon, "nom")) # ['Ali']
8 print(doublons(adherents, "nom")) # []
```

Propriété Test de cohérence. Un **test de cohérence** vérifie qu'une table respecte des contraintes attendues (types corrects, valeurs dans un intervalle plausible, absence de champ vide...). C'est une étape essentielle avant tout traitement automatisé de données réelles, souvent imparfaites.

CODE DE RÉFÉRENCE — Test de cohérence simple

```
1 def table_cohérente(table):
2     """Vérifie que chaque age est un entier positif raisonnable (0 à 120)."""
3     for ligne in table:
4         if not isinstance(ligne["age"], int) or not (0 ≤ ligne["age"] ≤ 120):
5             return False
6     return True
7
8 print(table_cohérente(adherents)) # True
9 print(table_cohérente(adherents + [{"nom": "X", "age": -5,
10     "categorie": "junior", "cotisation": 80}])))
11 # False
```

ERREUR FRÉQUENTE

Rechercher des doublons en comparant chaque ligne **entière** (`ligne1 == ligne2`) alors que la consigne porte sur une seule colonne (comme `nom`). Deux lignes peuvent partager le même `nom` sans être identiques sur les autres colonnes (âge différent par exemple) : il faut comparer uniquement la colonne concernée.

» **Python À la machine.** Ajoute une ligne dupliquée (même `nom` qu'une ligne existante, mais `age` différent) à la table `adherents`. Utilise `doublons` pour la détecter, puis écris une fonction `supprimer_doublons(table, colonne)` qui ne garde que la première occurrence de chaque valeur de colonne.

3.3 Trier une table

Propriété Tri suivant une colonne. Trier une table suivant une colonne consiste à réordonner ses lignes selon les valeurs de cette colonne (ordre croissant ou décroissant). En Python, la fonction `sorted` accepte un paramètre `key` : une fonction qui, pour chaque ligne, renvoie la valeur à utiliser pour le tri.

CODE DE RÉFÉRENCE — Trier une table avec `sorted`

```
1 adherents = [
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 ]
8
9 tries_par_age = sorted(adherents, key=lambda ligne: ligne["age"])
10 print([ligne["nom"] for ligne in tries_par_age])
11 # ['Sami', 'Ali', 'Nour', 'Karim', 'Yasmine']
```

Propriété Ordre décroissant. Pour trier en ordre **décroissant**, on ajoute le paramètre `reverse=True` à `sorted`.

Exemple. Trier les adhérents par cotisation décroissante :

ERREUR FRÉQUENTE

Écrire une fonction de tri « à la main » (par insertion ou sélection, vues au chapitre d'algorithmique) alors qu'une fonction intégrée existe et suffit largement. Le programme officiel autorise explicitement l'usage de `sorted` pour trier une table : implémenter son propre tri n'apporte rien ici et augmente le risque d'erreur.

DÉFINITION D3

`sorted(table, key=...)` renvoie une **nouvelle** liste triée, sans modifier `table`. La méthode `table.sort(key=...)`, elle, trie la liste **sur place** (elle modifie `table` et ne renvoie rien).

» **Python À la machine.** Trie la table `adherents` par ordre alphabétique du nom, puis par age décroissant. Vérifie, en affichant `adherents` avant et après un appel à `sorted` (pas `.sort()`), que la table d'origine n'est pas modifiée.

3.4 Fusionner deux tables

DÉFINITION D4

Le **domaine de valeurs** d'une colonne est l'ensemble des valeurs qu'elle peut prendre. Pour **fusionner** deux tables, on les combine en s'appuyant sur une colonne commune (une **clé**) dont le domaine de valeurs permet de faire correspondre les lignes de l'une avec celles de l'autre.

Exemple. On dispose d'une seconde table, les présences aux entraînements, partageant la colonne nom avec la table adherents :

```
1 presences = [
2     {"nom": "Ali", "seances": 12},
3     {"nom": "Sami", "seances": 9},
4     {"nom": "Yasmine", "seances": 15},
5 ]
```

Propriété Fusion par clé commune. Fusionner deux tables table1 et table2 suivant une colonne cle consiste à construire une nouvelle table où chaque ligne combine les colonnes des deux tables, pour les valeurs de cle présentes **dans les deux**. Une valeur de cle présente dans une seule des deux tables n'apparaît pas dans le résultat (elle est hors du domaine de valeurs commun).

CODE DE RÉFÉRENCE — Fusionner deux tables par une colonne commune

```
1 def fusionner(table1, table2, cle):
2     """Fusionne table1 et table2 sur les valeurs communes de 'cle'.
3
4     Precondition : 'cle' est une colonne presente dans les deux tables.
5     """
6     index2 = {ligne[cle]: ligne for ligne in table2}
7     fusion = []
8     for ligne1 in table1:
9         if ligne1[cle] in index2:
10             complement = {k: v for k, v in index2[ligne1[cle]].items() if k != cle}
11
12             fusion.append(**ligne1, **complement)
13     return fusion
14
15 adherents = [
16     {"nom": "Ali", "age": 16}, {"nom": "Sami", "age": 15}, {"nom": "Yasmine", "age": 22},
17 ]
18 resultat = fusionner(adherents, presences, "nom")
19 print(resultat)
20 # [{'nom': 'Ali', 'age': 16, 'seances': 12},
21 #  {'nom': 'Sami', 'age': 15, 'seances': 9},
22 #  {'nom': 'Yasmine', 'age': 22, 'seances': 15}]
```

Exemple. Si presences ne contient pas d'entrée pour "Karim" et "Nour" (absents du domaine de valeurs commun), ces adhérents **n'apparaissent pas** dans la table fusionnée, même s'ils sont présents dans adherents :

ERREUR FRÉQUENTE

Croire que la fusion garde **toutes** les lignes des deux tables. Par défaut, seules les lignes dont la clé est présente **dans les deux** tables sont conservées (fusion sur le domaine de valeurs **commun**) : c'est le comportement attendu par le programme officiel, qui met l'accent sur cette notion de domaine de valeurs.

» **Python À la machine.** Ajoute une troisième table cotisations_payees (colonnes nom et montant_paye) avec seulement 3 des 5 adhérents. Fusionne-la avec adherents via fusionner, et vérifie que la table résultante ne contient que les adhérents présents dans les deux tables.

Exercice 1

12 min

Une librairie stocke son inventaire dans le fichier livres.csv :

```
titre,auteur,prix,stock
Le Petit Prince,Saint-Exupéry,7.5,12
1984,Orwell,9.9,5
Le Comte de Monte-Cristo,Dumas,12.0,3
```

1. Écrire le code Python (avec le module csv) qui importe ce fichier dans une table livres.
2. Après import, quel est le type Python de livres[0][“prix”] ? Écrire le code qui convertit les colonnes prix (en float) et stock (en int) pour toutes les lignes.
3. En déduire la valeur totale du stock (somme de prix * stock pour chaque livre).

Exercice 2

12 min

On dispose de la table notes d'un contrôle continu :

```
1 notes = [
2     {"nom": "Amine", "matiere": "Maths", "note": 14},
3     {"nom": "Amine", "matiere": "NSI", "note": 16},
4     {"nom": "Lina", "matiere": "Maths", "note": 12},
5     {"nom": "Lina", "matiere": "NSI", "note": 18},
6     {"nom": "Omar", "matiere": "Maths", "note": 9},
7     {"nom": "Omar", "matiere": "NSI", "note": 11},
8 ]
```

1. Écrire une compréhension de liste renvoyant les lignes où la matière est “NSI” et la note est supérieure ou égale à 15.
2. Combien de lignes obtient-on ? Donner les noms correspondants.
3. Écrire une compréhension de liste renvoyant les noms des élèves ayant une note strictement inférieure à 10 (peu importe la matière).

Exercice 3

12 min

On dispose de la table commandes suivante :

```
1 commandes = [
2     {"id": "C001", "client": "Ali", "quantite": 3},
3     {"id": "C002", "client": "Sara", "quantite": 1},
4     {"id": "C001", "client": "Ali", "quantite": 3},
5     {"id": "C003", "client": "Nadia", "quantite": -2},
6 ]
```

1. Écrire une fonction `doublons(table, colonne)` qui renvoie les valeurs de colonne apparaissant plus d'une fois. L'appliquer à la colonne `id` : que trouve-t-on, et pourquoi est-ce a priori anormal pour un identifiant de commande ?
2. Écrire une fonction `coherente(table)` qui renvoie `False` si au moins une quantité n'est pas un entier strictement positif.
3. Identifier la ou les ligne(s) responsable(s) de l'incohérence détectée à la question 2.

Exercice 4

On reprend la table `notes` de l'exercice précédent.

1. Écrire l'instruction qui trie cette table par note **décroissante**.
2. Quelle est la première ligne de la table triée ? La dernière ?
3. La table `notes` d'origine est-elle modifiée par cette instruction ? Justifier en une phrase.

Exercice 5

Un lycée dispose de deux tables :

```

1  eleves = [
2      {"nom": "Amine", "classe": "1A"},
3      {"nom": "Lina", "classe": "1B"},
4      {"nom": "Omar", "classe": "1A"},
5      {"nom": "Sara", "classe": "1B"},
6  ]
7  absences = [
8      {"nom": "Amine", "nb_absences": 2},
9      {"nom": "Omar", "nb_absences": 0},
10     {"nom": "Sara", "nb_absences": 5},
11 ]

```

1. En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner `eleves` et `absences` sur la colonne `nom`. Combien de lignes contient le résultat ?
2. Quel élève de `eleves` n'apparaît **pas** dans la table fusionnée ? Pourquoi ?
3. Le CPE souhaite une table listant **tous** les élèves, y compris ceux sans donnée d'absence (avec `nb_absences` à 0 par défaut dans ce cas). Proposer, en français ou en pseudo-code, une modification de la fonction `fusionner` permettant d'obtenir ce résultat.

QCM d'auto-évaluation — Traitement de données en tables

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Import de table

1. [Q1] Après import d'un fichier CSV avec `csv.DictReader`, une valeur numérique comme "16" est de type :
 - A. `int`
 - B. `float`
 - C. `str`
 - D. `bool`

C2 — Recherche, doublons, cohérence

2. [Q2] Pour rechercher les lignes d'une table vérifiant un critère, l'outil le plus adapté en Python est :
 - A. une compréhension de liste avec condition.
 - B. une modification directe du fichier CSV.
 - C. la fonction `sorted`.
 - D. l'opérateur `+`.
3. [Q3] Pour détecter un doublon sur la colonne `id` d'une table, il faut :
 - A. comparer chaque ligne entière avec toutes les autres.
 - B. compter les occurrences de chaque valeur de la colonne `id` uniquement.
 - C. trier la table par ordre alphabétique.
 - D. supprimer toutes les lignes identiques.

C3 — Tri

4. [Q4] `sorted(table, key=lambda ligne: ligne["age"])` :
 - A. modifie `table` sur place et ne renvoie rien.
 - B. renvoie une nouvelle liste triée, sans modifier `table`.
 - C. provoque une erreur si `table` contient des dictionnaires.
 - D. trie uniquement les nombres, jamais les chaînes.

C4 — Fusion de tables

5. [Q5] Lors de la fusion de deux tables sur une colonne commune, une ligne de `table1` dont la clé n'existe pas dans `table2` :
 - A. provoque une erreur du programme.
 - B. apparaît dans le résultat avec des valeurs vides pour les colonnes de `table2`.
 - C. n'apparaît pas dans le résultat, par défaut.
 - D. est dupliquée dans le résultat.
6. [Q6] Le « domaine de valeurs » d'une colonne désigne :
 - A. le nombre de lignes de la table.
 - B. l'ensemble des valeurs que peut prendre cette colonne.
 - C. le type Python de la colonne uniquement.
 - D. le nom de la colonne dans le fichier CSV.

4 Langages et programmation

4.1 Constructions élémentaires

DÉFINITION D1

Les **constructions élémentaires** d'un langage de programmation impératif sont : la **séquence** (exécuter des instructions les unes après les autres), l'**affectation** (donner une valeur à une variable), la **conditionnelle** (if/else), les **boucles** (**bornées**, for, dont le nombre de répétitions est connu à l'avance ; **non bornées**, while, qui se répètent tant qu'une condition est vraie), et l'**appel de fonction**.

Exemple. La fonction somme_chiffres illustre ces cinq constructions :

CODE DE RÉFÉRENCE — Somme des chiffres d'un entier (toutes les constructions de base)

```
1 def somme_chiffres(n):
2     """Renvoie la somme des chiffres de l'entier positif n."""
3     total = 0                # affectation
4     while n > 0:              # boucle NON bornée (while)
5         chiffre = n % 10
6         total = total + chiffre # séquence d'instructions
7         n = n // 10
8     return total
9
10 for valeur in range(3):      # boucle bornée (for)
11     print(valeur, somme_chiffres(valeur * 100 + 34)) # appel de fonction
```

Propriété Conditionnelle. L'instruction if ... elif ... else ... exécute un bloc d'instructions différent selon qu'une (ou plusieurs) condition(s) booléenne(s) est/sont vraie(s).

```
1 def est_pair(n):
2     if n % 2 == 0:
3         return True
4     else:
5         return False
```

ERREUR FRÉQUENTE

Confondre boucle **bornée** (for, nombre d'itérations connu à l'avance, par exemple for i in range(10)) et boucle **non bornée** (while, le nombre d'itérations dépend de l'évolution d'une condition et n'est pas toujours connu à l'avance). Utiliser une boucle while sans faire évoluer la condition qu'elle teste crée une **boucle infinie**.

Malgré leur grande diversité, presque tous les langages de programmation partagent le même petit noyau de constructions de base.

» **Python À la machine.** Écris une fonction `compte_a_rebours(n)` qui affiche les entiers de n à 0 en utilisant une boucle `while` (boucle non bornée), puis une version `compte_a_rebours_for(n)` utilisant `for` (boucle bornée, avec `range` et un pas négatif). Compare les deux versions : laquelle te semble la plus naturelle ici ?

4.2 Diversité et unité des langages de programmation

Propriété Unité. Tous les langages de programmation impératifs partagent le même noyau de **constructions élémentaires** (§0.1) : c'est ce qui permet de reconnaître un algorithme même écrit dans un langage inconnu. Ce qui varie, ce sont les **traits particuliers** : syntaxe, typage, gestion des blocs d'instructions.

Exemple. La même fonction, « renvoyer le plus grand de deux nombres », écrite en Python puis dans un langage de style C :

```
1 def maximum(a, b):
2     if a > b:
3         return a
4     else:
5         return b
```

```
int maximum(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

Propriété Traits communs et traits particuliers. **Traits communs** aux deux écritures ci-dessus : une fonction nommée `maximum` prenant deux paramètres, une conditionnelle `if/else`, un `return` dans chaque branche. **Traits particuliers** au langage de style C : les types sont déclarés explicitement (`int`), les blocs d'instructions sont délimités par des accolades `{}` et non par l'indentation, chaque instruction se termine par un point-virgule.

DÉFINITION D2

Un langage est dit à **typage explicite** si le programmeur doit indiquer le type de chaque variable (comme en C ou en Java) ; il est à **typage implicite** (ou dynamique) si l'interpréteur déduit le type automatiquement (comme en Python).

Exemple. En Python, l'indentation **délimite** les blocs (elle fait partie de la syntaxe) ; dans un langage de style C, ce sont les accolades qui délimitent les blocs, et l'indentation n'est qu'une convention de lisibilité (le programme fonctionnerait même mal indenté).

ERREUR FRÉQUENTE

Croire qu'un algorithme « appartient » à un langage précis. Un même algorithme (par exemple « chercher le maximum d'une liste ») peut s'écrire dans n'importe quel langage Turing-complet : c'est la **traduction** qui change, pas l'algorithme lui-même.

» **Python À la machine.** Recherche (dans la documentation d'un langage que tu ne connais pas, comme JavaScript ou Scratch) comment s'écrit une boucle for et une conditionnelle if. Identifie les traits communs avec Python et les traits particuliers à ce langage.

4.3 Spécifier une fonction

DÉFINITION D3

Spécifier une fonction, c'est décrire précisément ce qu'elle fait **avant** de l'écrire : son **prototype** (nom, paramètres, valeur renvoyée), ses **préconditions** (ce que les arguments doivent vérifier pour que la fonction ait un sens) et ses **postconditions** (ce que le résultat doit vérifier).

Propriété Rôle des pré/postconditions. Une **précondition** est une hypothèse sur les **arguments**, que l'appelant doit garantir. Une **postcondition** est une garantie sur le **résultat**, que la fonction doit assurer si les préconditions sont respectées. En Python, on peut vérifier ces conditions avec l'instruction `assert`.

CODE DE RÉFÉRENCE — Spécification d'une fonction avec préconditions et postconditions

```
1 def aire_triangle(base, hauteur):
2     """Calcule l'aire d'un triangle à partir de sa base et sa hauteur.
3
4     Préconditions : base > 0 et hauteur > 0.
5     Postcondition : le resultat renvoyé est strictement positif.
6     """
7     assert base > 0, "la base doit être strictement positive"
8     assert hauteur > 0, "la hauteur doit être strictement positive"
9     aire = base * hauteur / 2
10    assert aire > 0
11    return aire
12
13 print(aire_triangle(6, 4)) # 12.0
```

Exemple. Appeler `aire_triangle(-2, 4)` viole la précondition `base > 0` : l'instruction `assert` interrompt le programme avec une `AssertionError` et le message associé, plutôt que de renvoyer un résultat absurde (une aire négative).

Propriété Le prototype. Le **prototype** d'une fonction résume sa signature : son nom, le nom et le rôle de chaque paramètre, et le type de la valeur renvoyée. En Python, on l'exprime généralement dans la première ligne de la **docstring** (chaîne de documentation entre triple guillemets), juste après la définition de la fonction.

ERREUR FRÉQUENTE

Écrire le code d'une fonction **avant** d'avoir réfléchi à ses préconditions. Sans préconditions explicites, une fonction peut recevoir des arguments absurdes (une base négative, une liste vide) et produire un résultat incorrect **sans qu'aucune erreur ne soit signalée** — le bug se manifeste alors bien plus tard, loin de sa cause réelle.

» **Python À la machine.** Écris une fonction `moyenne(notes)` qui calcule la moyenne d'une liste de notes. Réfléchis à sa précondition (la liste peut-elle être vide ?) et à sa postcondition (dans quel intervalle doit se situer le résultat, si les notes sont entre 0 et 20 ?). Ajoute les `assert` correspondants.

4.4 Mise au point de programmes

DÉFINITION D4

Un **jeu de tests** est un ensemble d'appels de fonction, chacun accompagné du résultat attendu, utilisé pour vérifier qu'un programme se comporte correctement. Mettre au point un programme, c'est faire évoluer son code jusqu'à ce qu'il réussisse tous les tests du jeu.

Exemple. Un élève écrit une fonction censée renvoyer le maximum d'une liste de nombres :

```
1 def maximum_bugue(liste):
2     maxi = 0
3     for x in liste:
4         if x > maxi:
5             maxi = x
6     return maxi
7
8 print(maximum_bugue([3, 7, 2]))    # 7 -- semble correct
```

Propriété Un jeu de tests réussi ne prouve pas la correction. Le succès d'un jeu de tests montre seulement que le programme fonctionne **sur les cas testés**. Il ne garantit **pas** que le programme est correct pour tous les cas possibles : un jeu de tests incomplet peut laisser passer un bug.

Contre-exemple. Le test précédent (liste de nombres positifs) réussit, mais `maximum_bugue` échoue sur une liste ne contenant que des nombres **négatifs** :

```
>>> maximum_bugue([-5, -1, -8])
0
```

ERREUR FRÉQUENTE

Initialiser un accumulateur de maximum à 0 (EF). C'est correct **si et seulement si** la liste contient au moins une valeur positive : sinon, 0 « gagne » à tort contre toutes les valeurs négatives. Il faut initialiser l'accumulateur avec le **premier élément** de la liste, pas avec une constante arbitraire.

CODE DE RÉFÉRENCE — Version corrigée, avec jeu de tests plus complet

```

1 def maximum(liste):
2     """Renvoie le plus grand element de 'liste'.
3
4     Precondition : 'liste' est non vide.
5     """
6     assert len(liste) > 0
7     maxi = liste[0]
8     for x in liste[1:]:
9         if x > maxi:
10            maxi = x
11    return maxi
12
13 # Jeu de tests couvrant plusieurs cas
14 assert maximum([3, 7, 2]) == 7          # cas "normal", positifs
15 assert maximum([-5, -1, -8]) == -1      # cas limite : tous negatifs
16 assert maximum([42]) == 42              # cas limite : un seul element

```

Propriété Concevoir un bon jeu de tests. Un jeu de tests efficace couvre : un cas « normal », un ou plusieurs **cas limites** (liste d'un seul élément, valeurs négatives, valeur nulle, chaîne vide...), et si pertinent un cas où une précondition est violée (pour vérifier qu'une erreur est bien signalée).

» **Python À la machine.** Écris une fonction `est_palindrome(mot)` qui teste si un mot se lit de la même façon à l'endroit et à l'envers. Écris un jeu de tests d'au moins 4 cas (un mot palindrome, un mot non palindrome, un mot d'une seule lettre, la chaîne vide) avant même d'écrire le code de la fonction, puis implémente-la jusqu'à ce que tous les tests passent.

4.5 Utiliser une bibliothèque

DÉFINITION D5

Une **bibliothèque** (ou *module*) est un ensemble de fonctions déjà écrites et testées, que l'on peut réutiliser avec l'instruction `import`. Utiliser une bibliothèque évite de « réinventer la roue » : c'est l'intérêt de la **modularisation**.

Propriété Lire la documentation. Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : son prototype (paramètres, valeur renvoyée), une description de son comportement, parfois des exemples. En Python, la fonction `help(...)` affiche la docstring d'une fonction, directement dans l'interpréteur.

Exemple. Le module `math` fournit des fonctions mathématiques prêtes à l'emploi :

```

1 import math
2
3 print(math.gcd(48, 18))    # 6 : plus grand commun diviseur
4 print(math.isqrt(50))     # 7 : partie entiere de la racine carree
5 print(math.ceil(4.2))     # 5 : arrondi a l'entier superieur
6 print(math.floor(4.8))    # 4 : arrondi a l'entier inferieur

```

```
>>> help(math.isqrt)
Help on built-in function isqrt in module math:

isqrt(n, /)
    Return the integer part of the square root of the input.
```

ERREUR FRÉQUENTE

Utiliser une fonction d'une bibliothèque **sans en lire la documentation**, en devinant son comportement. Par exemple, `math.isqrt(48)` renvoie la partie **entière tronquée** de la racine carrée (6, car $6^2 = 36 \leq 48 < 49 = 7^2$), ce qui n'est **pas** toujours la même chose qu'un **arrondi** : `round(math.sqrt(48))` vaut 7 (la racine exacte $\approx 6,93$ s'arrondit à 7), une valeur différente. Deux fonctions qui se ressemblent peuvent avoir des comportements différents : il faut vérifier dans la documentation.

Propriété Aucune connaissance exhaustive exigée. Le programme n'exige **aucune connaissance exhaustive** d'une bibliothèque particulière : savoir **chercher** et **lire** une documentation pour trouver la bonne fonction est plus important que de mémoriser toutes les fonctions disponibles.

» **Python À la machine.** Utilisez `help(str.split)` (ou consultez une documentation en ligne) pour comprendre ce que fait la méthode `split` des chaînes de caractères. Utilisez-la pour découper la chaîne "pomme,poire,banane" en une liste de trois mots, à partir de la virgule comme séparateur.

Exercice 1 10 min

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur `[1, 2, 3, 4, 5, 6]` (résultat attendu : 12) et sur `[1, 3, 5]` (résultat attendu : 0).

Exercice 2 12 min

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 3

⌚ 12 min

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur `largeur` et `longueur`, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 4

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```

1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False

```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 5

On souhaite calculer la moyenne et la médiane de la liste de notes [10, 12, 14, 20], sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

QCM d'auto-évaluation — Langages et programmation

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Constructions élémentaires

1. [Q1] Une boucle `while` est dite « non bornée » car :
 - A. elle ne peut jamais se terminer.
 - B. le nombre d'itérations n'est pas forcément connu à l'avance.
 - C. elle ne contient jamais de condition.
 - D. elle est plus rapide qu'une boucle `for`.

C2 — Diversité et unité des langages

2. [Q2] Ce qui varie principalement d'un langage de programmation impératif à un autre, c'est :
- A. le noyau de constructions élémentaires (qui est différent à chaque fois).
 - B. la syntaxe et des traits particuliers comme le typage.
 - C. le fait qu'un même algorithme ne puisse s'écrire que dans un seul langage.
 - D. rien : tous les langages sont strictement identiques.

C3 — Spécification

3. [Q3] Une précondition d'une fonction porte sur :
- A. le résultat renvoyé par la fonction.
 - B. les arguments passés à la fonction.
 - C. le nom de la fonction.
 - D. le nombre de lignes de code de la fonction.

C4 — Mise au point

4. [Q4] Un jeu de tests entièrement réussi :
- A. prouve que le programme est correct dans tous les cas.
 - B. montre seulement que le programme fonctionne sur les cas testés.
 - C. garantit l'absence de tout bug futur.
 - D. n'a aucun intérêt si le programme compile.

C5 — Bibliothèques

5. [Q5] Avant d'utiliser une fonction inconnue d'une bibliothèque, il est recommandé de :
- A. deviner son comportement à partir de son nom.
 - B. consulter sa documentation (docstring, `help(...)`).
 - C. recopier son code source avant de l'utiliser.
 - D. l'essayer uniquement sur des cas limites.
6. [Q6] Le programme officiel de NSI Première, concernant les bibliothèques, exige :
- A. une connaissance exhaustive d'au moins une bibliothèque.
 - B. de savoir chercher et lire une documentation pour trouver la bonne fonction.
 - C. de ne jamais utiliser de bibliothèque externe.
 - D. de mémoriser toutes les fonctions du module `math`.

5 Algorithmique 1 : parcours et tris

5.1 Parcours séquentiel d'un tableau

DÉFINITION D1

Un **parcours séquentiel** examine les éléments d'un tableau **un par un**, dans l'ordre, généralement à l'aide d'une boucle for. De nombreux algorithmes de base (recherche, extremum, moyenne) reposent sur ce principe.

Propriété Recherche d'une occurrence. Rechercher une occurrence d'une valeur dans un tableau consiste à parcourir le tableau et à s'arrêter (en renvoyant l'indice trouvé) dès que la valeur est rencontrée. Si elle n'apparaît nulle part, on renvoie une valeur conventionnelle (par exemple -1).

CODE DE RÉFÉRENCE — Recherche d'une occurrence (coût linéaire)

```
1 def recherche_occurrence(tableau, valeur):
2     """Renvoie l'indice de la première occurrence de 'valeur', ou -1 si absente."""
3
4     for i in range(len(tableau)):
5         if tableau[i] == valeur:
6             return i
7     return -1
8
9 print(recherche_occurrence([5, 3, 8, 1], 8)) # 2
10 print(recherche_occurrence([5, 3, 8, 1], 99)) # -1
```

Propriété Coût de la recherche. Dans le **pire cas** (valeur absente, ou en dernière position), la recherche d'occurrence examine **tous** les éléments du tableau : son coût est **linéaire** (proportionnel à la taille n du tableau, noté $O(n)$).

Propriété Recherche d'un extremum. Rechercher le maximum (ou le minimum) d'un tableau non vide consiste à parcourir ses éléments en conservant, dans un accumulateur, la plus grande (ou plus petite) valeur rencontrée jusque-là. L'accumulateur doit être initialisé avec le **premier élément** du tableau, jamais avec une constante arbitraire.

CODE DE RÉFÉRENCE — Recherche du maximum et calcul d'une moyenne

```
1 def maximum(tableau):
2     """Renvoie le plus grand élément de 'tableau' (non vide)."""
3     maxi = tableau[0]
```

```

4     for x in tableau[1:]:
5         if x > maxi:
6             maxi = x
7     return maxi
8
9     def moyenne(tableau):
10        """Renvoie la moyenne des elements de 'tableau' (non vide)."""
11        return sum(tableau) / len(tableau)
12
13    print(maximum([5, 3, 8, 1])) # 8
14    print(moyenne([5, 3, 8, 1])) # 4.25

```

ERREUR FRÉQUENTE

Initialiser l'accumulateur du maximum à 0 (EF). Cette erreur produit un résultat incorrect dès que **tous** les éléments du tableau sont négatifs : le maximum trouvé serait alors 0, une valeur absente du tableau. Il faut toujours initialiser l'accumulateur avec le **premier élément** du tableau.

» **Python À la machine.** Écris une fonction recherche_toutes_occurrences(tableau, valeur) qui renvoie la liste de **tous** les indices où valeur apparaît dans tableau (et non seulement le premier). Teste-la sur [3, 7, 3, 3, 9] avec valeur=3 : le résultat attendu est [0, 2, 3].

5.2 Le tri par insertion

DÉFINITION D2

Le **tri par insertion** construit progressivement une partie triée du tableau, en **insérant**, un par un, chaque élément restant à sa bonne place dans la partie déjà triée (comme on trie des cartes à jouer en main, une par une).

CODE DE RÉFÉRENCE — Tri par insertion

```

1     def tri_insertion(tableau):
2         """Trie 'tableau' par ordre croissant, par insertion (renvoie une copie triee).
3         """
4         tableau = tableau[:]
5         for i in range(1, len(tableau)):
6             valeur = tableau[i]
7             j = i - 1
8             while j ≥ 0 and tableau[j] > valeur:
9                 tableau[j + 1] = tableau[j]
10                j -= 1
11                tableau[j + 1] = valeur
12        return tableau
13    print(tri_insertion([5, 3, 8, 1, 9, 2])) # [1, 2, 3, 5, 8, 9]

```

Propriété Termination et coût. La boucle for externe s'exécute un nombre **fixe** de fois ($n - 1$ itérations pour un tableau de taille n) : elle termine nécessairement. La boucle while interne termine car j **décroît strictement** à chaque tour et est bornée par 0. Dans le **pire cas** (tableau trié en ordre décroissant), le coût est **quadratique** ($O(n^2)$) : chaque insertion peut nécessiter de décaler tous les éléments déjà triés.

Démonstration. **Invariant de boucle** : au début de chaque itération i de la boucle for (avant l'insertion de `tableau[i]`), le sous-tableau `tableau[0:i]` est **trié** (et contient les mêmes éléments que le sous-tableau original `tableau[0:i]` avant tout tri).

Initialisation. Avant la première itération ($i = 1$), `tableau[0:1]` est un tableau à un seul élément : il est trivialement trié.

Conservation. Supposons `tableau[0:i]` trié au début de l'itération i . La boucle while interne décale vers la droite tous les éléments de `tableau[0:i]` strictement supérieurs à `valeur = tableau[i]`, puis place `valeur` à la position ainsi libérée. Le sous-tableau `tableau[0:i+1]` obtenu est donc trié : l'invariant est conservé pour l'itération suivante.

Termination. Après la dernière itération ($i = n - 1$), l'invariant donne : `tableau[0:n]` (le tableau entier) est trié. L'algorithme est donc **correct**.

ERREUR FRÉQUENTE

Croire qu'un invariant de boucle doit être vérifié **une seule fois** (par exemple seulement à la fin). Un invariant doit être vrai **à chaque** itération : c'est cette répétition, de l'initialisation jusqu'à la terminaison, qui constitue la preuve par récurrence de la correction de l'algorithme.

» **Python À la machine.** Ajoute des instructions `assert tableau[0:i] == sorted(tableau[0:i])` dans ta propre implémentation du tri par insertion, juste avant l'insertion de chaque nouvel élément, pour vérifier expérimentalement l'invariant sur plusieurs tableaux de ton choix.

5.3 Le tri par sélection

DÉFINITION D3

Le **tri par sélection** construit progressivement une partie triée en **choisissant**, à chaque étape, le plus petit élément **restant** et en le plaçant à la bonne position (au lieu d'insérer un élément dans la partie triée, comme pour le tri par insertion).

CODE DE RÉFÉRENCE — Tri par sélection

```
1 def tri_selection(tableau):
2     """Trie 'tableau' par ordre croissant, par selection (renvoie une copie triee).
3     """
4     tableau = tableau[:]
5     n = len(tableau)
6     for i in range(n - 1):
7         indice_min = i
8         for j in range(i + 1, n):
9             if tableau[j] < tableau[indice_min]:
10                 indice_min = j
11         tableau[i], tableau[indice_min] = tableau[indice_min], tableau[i]
```

```

11     return tableau
12
13     print(tri_selection([5, 3, 8, 1, 9, 2])) # [1, 2, 3, 5, 8, 9]

```

Propriété Terminaison et coût. Les deux boucles for (externe et interne) s'exécutent un nombre **fixe** de fois, déterminé à l'avance : l'algorithme termine nécessairement. Dans **tous** les cas (pas seulement le pire), le coût est **quadratique** ($O(n^2)$) : contrairement au tri par insertion, la recherche du minimum restant parcourt toujours l'intégralité de la partie non triée, même si le tableau est déjà trié.

Démonstration. Invariant de boucle : au début de chaque itération i de la boucle for externe, le sous-tableau `tableau[0:i]` est **trié**, et chacun de ses éléments est **inférieur ou égal** à tous les éléments de `tableau[i:n]`.

Initialisation. Avant la première itération ($i = 0$), `tableau[0:0]` est le tableau vide : l'invariant est trivialement vrai.

Conservation. Supposons l'invariant vrai au début de l'itération i . La boucle for interne détermine `indice_min`, l'indice du plus petit élément de `tableau[i:n]`. L'échange place cet élément minimal en position i . Comme `tableau[0:i]` était déjà trié et que ses éléments étaient tous inférieurs ou égaux à `tableau[i:n]` (donc au minimum choisi), `tableau[0:i+1]` est trié ; et ce minimum étant le plus petit du reste, tous les éléments de `tableau[0:i+1]` restent inférieurs ou égaux à `tableau[i+1:n]`. L'invariant est conservé.

Terminaison. Après la dernière itération ($i = n - 2$), l'invariant donne : `tableau[0:n-1]` est trié, avec son dernier élément inférieur ou égal à `tableau[n-1:n]` : le tableau entier `tableau[0:n]` est donc trié.

Propriété Insertion contre sélection. Les deux tris ont le même coût dans le pire cas ($O(n^2)$), mais le tri par **insertion** est plus rapide en pratique sur un tableau **presque trié** (la boucle while interne s'arrête vite), alors que le tri par **sélection** effectue toujours le même nombre de comparaisons, quel que soit l'état initial du tableau.

ERREUR FRÉQUENTE

Confondre les deux tris : dans le tri par sélection, on **recherche le minimum** de toute la partie restante avant de l'échanger ; dans le tri par insertion, on **insère** l'élément courant à sa place en décalant les éléments plus grands. Ce ne sont pas les mêmes opérations, même si le résultat final est identique.

» **Python À la machine.** Compare expérimentalement le nombre de comparaisons effectuées par `tri_insertion` et `tri_selection` sur un tableau **déjà trié** de 20 éléments (ajoute un compteur global incrémenté à chaque comparaison dans les deux fonctions). Le résultat confirme-t-il la propriété du cours ?

Exercice 1 10 min

On donne le tableau de notes `[12, 4, 9, 1, 7]`.

1. Écrire une fonction `minimum(tableau)` qui renvoie le plus petit élément d'un tableau non vide (sur le modèle de `maximum` du cours).
2. Calculer la note minimale et la moyenne de ce tableau.

- Un élève initialise $mini = 20$ avant sa boucle, en argumentant que 20 est la note maximale possible. Pourquoi ce choix, bien que semblant raisonnable ici, est-il une mauvaise pratique en général ?

Exercice 2

La fonction `recherche_occurrence` du cours ne renvoie que le premier indice trouvé.

- Écrire une fonction `recherche_toutes_occurrences(tableau, valeur)` qui renvoie la liste de **tous** les indices où `valeur` apparaît.
- Tester cette fonction sur `[3, 7, 3, 3, 9]` avec `valeur=3`, puis avec `valeur=5`.
- Quel est le coût, dans le pire cas, de cette fonction (en fonction de la taille n du tableau) ? Est-il différent de celui de `recherche_occurrence` ? Justifier.

Exercice 3

On applique le tri par insertion au tableau `[4, 2, 7, 1]`.

- Dérouler l'algorithme : donner le contenu du tableau après chaque itération de la boucle `for` externe ($i = 1, i = 2, i = 3$).
- Pour chaque étape, vérifier que le sous-tableau `tableau[0:i+1]` est bien trié (c'est l'invariant de boucle du cours).
- Le tableau final est-il correctement trié ? Vérifier avec le code.

Exercice 4

On applique le tri par sélection au tableau `[6, 3, 8, 2]`.

- Dérouler l'algorithme : pour chaque itération de la boucle `for` externe ($i = 0, i = 1, i = 2$), donner l'indice du minimum trouvé dans la partie restante, et le contenu du tableau après l'échange.
- Pour chaque étape, vérifier que `tableau[0:i+1]` est trié **et** que son maximum est inférieur ou égal au minimum de `tableau[i+1:]` (c'est l'invariant de boucle du cours).
- Le tableau final est-il correctement trié ?

Exercice 5

On ajoute un compteur de comparaisons aux deux fonctions de tri du cours (une variable incrémentée à chaque comparaison entre deux éléments du tableau).

- Sur un tableau **déjà trié** de 20 éléments (`list(range(1, 21))`), combien de comparaisons effectue le tri par insertion ? Le tri par sélection ?
- Ce résultat confirme-t-il la propriété du cours (« insertion contre sélection ») ? Expliquer pourquoi le tri par insertion effectue si peu de comparaisons dans ce cas précis.
- Sur un tableau trié en ordre **strictement décroissant** de 20 éléments, on obtient 190 comparaisons pour le tri par insertion **et** 190 pour le tri par sélection. Ce résultat te surprend-il ? Explique, en raisonnant sur le déroulement de la boucle `while` du tri par insertion dans ce cas précis, pourquoi son avantage habituel disparaît ici.

QCM d'auto-évaluation — Algorithmique 1 : parcours et tris

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Recherche d'occurrence

- [Q1] Le coût, dans le pire cas, d'une recherche séquentielle d'une valeur dans un tableau de taille n est :

- A. constant.
- B. logarithmique.
- C. linéaire, $O(n)$.
- D. quadratique, $O(n^2)$.

C2 — Extremum

2. [Q2] Pour rechercher le maximum d'un tableau non vide, il faut initialiser l'accumulateur avec :
- A. la valeur 0.
 - B. le premier élément du tableau.
 - C. la plus grande valeur possible.
 - D. la dernière valeur du tableau.

C3/C4 — Tri par insertion

3. [Q3] Le tri par insertion construit la partie triée du tableau en :
- A. recherchant le minimum de la partie restante à chaque étape.
 - B. insérant chaque nouvel élément à sa bonne place dans la partie déjà triée.
 - C. triant d'abord la seconde moitié du tableau.
 - D. échangeant des éléments au hasard.
4. [Q4] L'invariant de boucle du tri par insertion affirme qu'au début de l'itération i :
- A. le tableau entier est déjà trié.
 - B. le sous-tableau `tableau[0:i]` est trié.
 - C. le sous-tableau `tableau[i:n]` est trié.
 - D. aucune partie du tableau n'est triée.

C5/C6 — Tri par sélection

5. [Q5] Le tri par sélection construit la partie triée du tableau en :
- A. insérant chaque élément à sa bonne place.
 - B. recherchant le minimum de la partie restante et l'échangeant à sa place.
 - C. divisant le tableau en deux moitiés récursivement.
 - D. ne comparant jamais deux éléments.
6. [Q6] Contrairement au tri par insertion, le tri par sélection effectue, dans TOUS les cas (tableau déjà trié ou non) :
- A. un nombre de comparaisons variable selon l'ordre initial.
 - B. toujours le même nombre de comparaisons, quel que soit l'ordre initial.
 - C. zéro comparaison.
 - D. un nombre de comparaisons linéaire, $O(n)$.

6 Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins

6.1 Recherche dichotomique

DÉFINITION D1

La **recherche dichotomique** recherche une valeur dans un tableau **trié** en comparant systématiquement la valeur cherchée à l'élément **du milieu** de la zone de recherche restante, et en ne conservant que la moitié où la valeur peut encore se trouver.

| Chercher un mot dans un dictionnaire papier trié : on ouvre au milieu, on compare, et on ne garde que la moitié pertinente. C'est le principe de la **dichotomie**.

CODE DE RÉFÉRENCE — Recherche dichotomique

```
1 def recherche_dichotomique(tableau_trie, valeur):
2     """Renvoie l'indice de 'valeur' dans 'tableau_trie' (trie), ou -1 si absente.
3     """
4     borne_inf, borne_sup = 0, len(tableau_trie) - 1
5     while borne_inf <= borne_sup:
6         milieu = (borne_inf + borne_sup) // 2
7         if tableau_trie[milieu] == valeur:
8             return milieu
9         elif tableau_trie[milieu] < valeur:
10            borne_inf = milieu + 1
11        else:
12            borne_sup = milieu - 1
13    return -1
14
15 t = [1, 3, 5, 7, 9, 11, 13, 15]
16 print(recherche_dichotomique(t, 11)) # 5
17 print(recherche_dichotomique(t, 4)) # -1
```

Propriété Coût de la dichotomie. À chaque itération, la zone de recherche est **divisée par deux**. Le coût de la recherche dichotomique est donc **logarithmique** ($O(\log_2 n)$), bien plus rapide que le coût linéaire de la recherche séquentielle pour un grand tableau.

Démonstration. **Terminaison de la recherche dichotomique.**

On considère le **variant de boucle** $V = \text{borne_sup} - \text{borne_inf}$.

V est bien défini et positif ou nul tant que la boucle continue, puisque la condition de la boucle while est précisément $\text{borne_inf} \leq \text{borne_sup}$.

V **décroît strictement** à chaque itération : si $\text{tableau_trie}[\text{milieu}] < \text{valeur}$, alors borne_inf devient $\text{milieu} + 1$, strictement supérieur à son ancienne valeur (puisque $\text{milieu} \geq \text{borne_inf}$), donc V diminue. Si $\text{tableau_trie}[\text{milieu}] > \text{valeur}$, alors borne_sup devient $\text{milieu} - 1$, strictement inférieur à son ancienne valeur (puisque $\text{milieu} \leq \text{borne_sup}$), donc V diminue également.

Conclusion. V est un entier positif ou nul qui décroît strictement à chaque tour de boucle : une suite d'entiers positifs strictement décroissante est nécessairement **finie**. La boucle `while` termine donc en un nombre fini d'itérations.

ERREUR FRÉQUENTE

Appliquer la dichotomie sur un tableau **non trié**. La dichotomie repose entièrement sur le tri : comparer à l'élément du milieu ne permet d'éliminer une moitié **que si** on est certain que tous les éléments de cette moitié sont, eux aussi, inférieurs (ou supérieurs) à la valeur cherchée — ce qui n'est garanti que si le tableau est trié.

» **Python À la machine.** Modifie `recherche_dichotomique` pour qu'elle compte le nombre d'itérations effectuées, et compare ce nombre à $\log_2(n)$ pour un tableau trié de 1000 éléments (utilise `math.log2`).

6.2 Algorithmes gloutons

DÉFINITION D2

Un **algorithme glouton** construit une solution étape par étape, en faisant à chaque étape le choix qui semble **le meilleur sur le moment** (le choix « localement optimal »), sans jamais revenir en arrière.

Exemple. Le **rendu de monnaie** : pour rendre un montant donné avec le moins de pièces possible, on choisit à chaque étape la plus grande pièce (ou billet) ne dépassant pas le montant restant.

CODE DE RÉFÉRENCE — Rendu de monnaie glouton

```
1 def rendu_glouton(montant, pieces):
2     """Rend 'montant' en utilisant le moins de pieces possible (methode gloutonne).
3     """
4     pieces_triees = sorted(pieces, reverse=True)
5     rendu = []
6     for p in pieces_triees:
7         while montant >= p:
8             rendu.append(p)
9             montant -= p
10    return rendu
11 print(rendu_glouton(78, [50, 20, 10, 5, 2, 1])) # [50, 20, 5, 2, 1]
```

Propriété Un algorithme glouton n'est pas toujours optimal. Un algorithme glouton donne une solution **rapide à calculer**, mais qui n'est **pas garantie optimale** en général : le choix localement optimal à chaque étape peut conduire à une solution globalement moins bonne qu'une autre approche.

Contre-exemple. Avec le système de pièces $\{4, 3, 1\}$ (non standard), pour rendre 6 : la méthode gloutonne choisit d'abord 4 (la plus grande pièce ≤ 6), puis doit compléter les 2 restants avec deux pièces de 1 : elle utilise 3 pièces ($4 + 1 + 1$). Or une solution avec 2 pièces existe : $3 + 3 = 6$. L'algorithme glouton n'est donc, dans ce cas, **pas optimal**.

Propriété Quand l'approche gloutonne fonctionne-t-elle ? Pour le système de pièces en euros {1, 2, 5, 10, 20, 50, 100, 200} (centimes ou euros), la méthode gloutonne donne **toujours** le nombre minimal de pièces : ce système a une propriété mathématique particulière (chaque pièce est « assez grande » par rapport aux suivantes) qui garantit l'optimalité du choix glouton. Ce n'est pas le cas pour un système de pièces quelconque.

ERREUR FRÉQUENTE

Croire qu'un algorithme glouton donne toujours la meilleure solution possible. Un algorithme glouton est une **heuristique** (une méthode rapide et souvent efficace en pratique), pas une garantie d'optimalité : il faut vérifier, au cas par cas, si les propriétés du problème garantissent l'optimalité de l'approche gloutonne.

» **Python À la machine.** Teste `rendu_glouton` avec le système de pièces {1, 5, 6, 9} pour rendre 11. Combien de pièces la méthode gloutonne utilise-t-elle ? Trouve, en cherchant à la main, une solution utilisant moins de pièces.

6.3 L'algorithme des k plus proches voisins

DÉFINITION D3

L'algorithme des **k plus proches voisins** prédit la classe d'un nouvel élément en examinant les **k** éléments **déjà classés** les plus proches de lui (au sens d'une distance, par exemple la distance euclidienne), et en lui attribuant la classe **majoritaire** parmi ces **k** voisins.

Propriété Distance euclidienne entre deux points. Pour deux points (x_1, y_1) et (x_2, y_2) du plan, la distance euclidienne est $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

CODE DE RÉFÉRENCE — Algorithme des k plus proches voisins

```
1 def distance(p1, p2):
2     """Distance euclidienne entre deux points (x, y)."""
3     return ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5
4
5 def k_plus_proches_voisins(points_classes, nouveau_point, k):
6     """Prédit la classe de 'nouveau_point' à partir de ses k plus proches voisins.
7
8     'points_classes' : liste de triplets (x, y, classe) déjà étiquetés.
9     """
10    distances = [(distance((x, y), nouveau_point), classe)
11                  for (x, y, classe) in points_classes]
12    distances.sort(key=lambda d: d[0])
13    k_plus_proches = distances[:k]
14    classes = [c for (_, c) in k_plus_proches]
15    return max(set(classes), key=classes.count)
16
17 points = [
18     (1, 1, "A"), (2, 1, "A"), (1, 2, "A"),
19     (8, 8, "B"), (9, 8, "B"), (8, 9, "B"),
```

```

20 ]
21 print(k_plus_proches_voisins(points, (1.5, 1.5), 3)) # A
22 print(k_plus_proches_voisins(points, (8.5, 8.5), 3)) # B

```

Propriété Choix de k . Le choix du nombre k de voisins influence le résultat : un k **trop petit** rend la prédiction sensible au bruit (un seul voisin atypique peut fausser le résultat) ; un k **trop grand** risque d'inclure des voisins trop éloignés, peu pertinents pour la classification. On choisit souvent k impair (pour éviter les égalités entre deux classes) et adapté à la taille du jeu de données.

ERREUR FRÉQUENTE

Oublier de **trier** les distances avant de sélectionner les k plus proches voisins. Sans tri (par exemple en prenant les k **premiers** points de la liste `points_classes`, dans l'ordre où ils sont donnés), on ne sélectionne pas du tout les voisins les plus proches, mais des points arbitraires.

» **Python À la machine.** Ajoute un point mal classé aux données de l'exemple, par exemple (1, 1, "B") au milieu du groupe "A", puis reclasse le point (1.5, 1.5) avec $k = 3$ puis $k = 5$. Le résultat change-t-il ? Explique en quoi cela illustre l'intérêt d'un k pas trop petit face au bruit dans les données.

Exercice 1 12 min

On applique recherche_dichotomique du cours au tableau trié [2, 5, 8, 12, 16, 23, 38, 45, 56, 72] (indices 0 à 9), pour rechercher la valeur 23.

1. Pour chaque itération de la boucle while, donner les valeurs de borne_inf, borne_sup, milieu et tableau_trie[milieu].
2. Combien d'itérations sont nécessaires pour trouver la valeur 23 ? Quel indice est renvoyé ?
3. Calculer le variant de boucle $V = \text{borne_sup} - \text{borne_inf}$ au début de chaque itération. Vérifie qu'il décroît strictement.

Exercice 2 12 min

Un élève applique recherche_dichotomique du cours au tableau [23, 5, 42, 8, 16, 3, 99], qui n'est pas trié, pour chercher la valeur 23.

1. Que renvoie recherche_dichotomique([23, 5, 42, 8, 16, 3, 99], 23) ? La valeur 23 est-elle pourtant présente dans le tableau ?
2. Dérouler les deux premières itérations pour expliquer précisément pourquoi l'algorithme élimine à tort la portion du tableau contenant la valeur 23.
3. Quelle condition, énoncée dans le cours, ce tableau ne respecte-t-il pas ?

Exercice 3 12 min

1. Appliquer la méthode gloutonne du cours pour rendre 63 euros avec le système de pièces {50, 20, 10, 5, 2, 1}. Détailler le choix de chaque pièce.
2. Vérifier que la somme des pièces rendues vaut bien 63.
3. Ce système de pièces (euros) garantit-il que la méthode gloutonne donne toujours le nombre minimal de pièces ? Est-ce le cas pour **tout** système de pièces ? Justifier en citant le contre-exemple du cours.

Exercice 4



12 min

On dispose d'une base de fruits déjà classés, chacun décrit par sa masse (en g) et sa longueur (en cm) :

```
1 fruits = [
2     (150, 7, "Pomme"), (170, 7.5, "Pomme"), (140, 6.5, "Pomme"),
3     (120, 20, "Banane"), (130, 22, "Banane"), (110, 18, "Banane"),
4 ]
```

1. En utilisant `k_plus_proches_voisins` du cours, prédire la classe d'un nouveau fruit de masse 145 g et de longueur 8 cm, avec $k = 3$.
2. Cette prédiction te semble-t-elle raisonnable au vu des données (comparer masse et longueur du nouveau fruit à celles des deux groupes) ?
3. Que se passerait-il, à ton avis, si on utilisait $k = 6$ (tous les fruits de la base) au lieu de $k = 3$? La classe obtenue serait-elle nécessairement pertinente ?

Exercice 5



15 min

Le problème du **sac à dos** : on dispose d'objets, chacun avec un poids et une valeur, et d'un sac de capacité limitée. On veut maximiser la valeur totale des objets emportés sans dépasser la capacité. Une approche gloutonne classique choisit les objets par **rapport valeur/poids décroissant**.

```
1 def sac_a_dos_glouton(objets, capacite):
2     """objets : liste de (nom, poids, valeur)."""
3     objets_tries = sorted(objets, key=lambda o: o[2] / o[1], reverse=True)
4     charge = 0
5     valeur_totale = 0
6     choisis = []
7     for nom, poids, valeur in objets_tries:
8         if charge + poids <= capacite:
9             choisis.append(nom)
10            charge += poids
11            valeur_totale += valeur
12     return choisis, valeur_totale
```

On dispose des objets X (poids 6, valeur 30), Y (poids 5, valeur 20) et Z (poids 5, valeur 20), pour un sac de capacité 10.

1. Calculer le rapport valeur/poids de chaque objet, et en déduire l'ordre dans lequel l'algorithme les examine.
2. Exécuter `sac_a_dos_glouton` sur ces données. Quels objets sont choisis, et quelle est la valeur totale obtenue ?
3. Trouver, en cherchant à la main, une combinaison d'objets de poids total ≤ 10 offrant une valeur **strictement supérieure**. L'algorithme glouton est-il optimal dans ce cas ?

QCM d'auto-évaluation — Dichotomie, gloutons, k plus proches voisins

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Dichotomie

1. [Q1] La recherche dichotomique nécessite que le tableau soit :
A. de taille paire.

- B. trié.
 - C. composé uniquement d'entiers.
 - D. de taille inférieure à 100.
2. [Q2] Le variant de boucle $V = \text{borne_sup} - \text{borne_inf}$ de la recherche dichotomique sert à prouver :
- A. que l'algorithme trouve toujours la valeur cherchée.
 - B. que la boucle termine en un nombre fini d'itérations.
 - C. que le tableau est trié.
 - D. que le coût est linéaire.

C2 — Algorithmes gloutons

3. [Q3] Un algorithme glouton fait, à chaque étape, le choix :
- A. globalement optimal, garanti par calcul exhaustif.
 - B. localement optimal, sans revenir en arrière.
 - C. aléatoire.
 - D. identique au choix précédent.
4. [Q4] Avec le système de pièces $\{4, 3, 1\}$, rendre 6 de façon gloutonne donne :
- A. la solution optimale à 2 pièces.
 - B. une solution à 3 pièces, non optimale.
 - C. une erreur, car 6 n'est pas atteignable.
 - D. une solution à 6 pièces.

C3 — k plus proches voisins

5. [Q5] L'algorithme des k plus proches voisins prédit la classe d'un nouvel élément à partir de :
- A. la classe majoritaire de ses k plus proches voisins déjà classés.
 - B. la classe du point le plus éloigné.
 - C. une valeur aléatoire.
 - D. la moyenne de toutes les classes existantes.
6. [Q6] Choisir un k égal à la taille totale du jeu de données (tous les points) :
- A. est toujours la meilleure stratégie.
 - B. fait perdre le sens de la méthode, basée sur la proximité locale.
 - C. accélère toujours le calcul.
 - D. est obligatoire pour que l'algorithme fonctionne.

7 Interactions entre l'homme et la machine sur le Web

7.1 Composants graphiques et événements

DÉFINITION D1

Un **composant graphique** d'une page Web est un élément avec lequel l'utilisateur peut interagir : bouton (`<button>`), champ de texte (`<input type="text">`), case à cocher (`<input type="checkbox">`), liste déroulante (`<select>`), lien (`<a>`)...

Exemple. Extrait de code HTML décrivant un bouton :

```
<button id="bouton_valider">Valider</button>
```

Propriété Description vs comportement. Le code **HTML** décrit uniquement l'**apparence** et la **structure** d'un composant graphique (« il y a un bouton, avec ce texte »). Le **comportement** du composant (que se passe-t-il quand on clique dessus ?) est programmé séparément, en général en **JavaScript**.

DÉFINITION D2

Un **événement** est une action détectée par le navigateur (clic de souris, appui sur une touche, chargement de la page, soumission d'un formulaire...). Une **fonction associée à un événement** (un *gestionnaire d'événement*, ou *event handler*) est exécutée automatiquement lorsque cet événement se produit sur un composant donné.

Exemple. En JavaScript, on associe une fonction à l'événement « clic » d'un bouton :

```
document.getElementById("bouton_valider").addEventListener("click", function() {  
    alert("Formulaire valide !");  
});
```

Propriété Simuler un gestionnaire d'événements. Le principe d'un gestionnaire d'événements peut se modéliser simplement : un **dictionnaire** associe à chaque identifiant de composant la fonction à exécuter lors du clic. « Déclencher un clic » revient alors à rechercher puis appeler la fonction associée.

CODE DE RÉFÉRENCE — Modélisation Python d'un gestionnaire d'événements

```
1 gestionnaires = {}  
2
```

| Une page Web n
limite pas à du tex
statique : des
composants
graphiques perm
l'utilisateur d'inter
avec elle (cliquer,
du texte, choisir un
option).

```

3 def ajouter_gestionnaire(id_composant, fonction):
4     """Associe 'fonction' à l'événement clic du composant 'id_composant'."""
5     gestionnaires[id_composant] = fonction
6
7 def declencher_clic(id_composant):
8     """Simule un clic : exécute la fonction associée, si elle existe."""
9     if id_composant in gestionnaires:
10         return gestionnaires[id_composant]()
11     return None
12
13 compteur = {"valeur": 0}
14 def incrementer():
15     compteur["valeur"] += 1
16     return compteur["valeur"]
17
18 ajouter_gestionnaire("bouton_plus", incrementer)
19 print(declencher_clic("bouton_plus")) # 1
20 print(declencher_clic("bouton_plus")) # 2

```

ERREUR FRÉQUENTE

Confondre la **description** HTML d'un composant (son apparence, son identifiant) avec son **comportement** programmé (ce qui se passe lors d'un événement). Le code HTML `<button id="bouton_plus">+</button>` ne fait **rien** par lui-même tant qu'aucune fonction n'est associée à l'un de ses événements.

» **Python À la machine.** Modifie le code Python ci-dessus pour ajouter un second gestionnaire, associé à "bouton_moins", qui décrémente compteur["valeur"]. Déclenche successivement "bouton_plus", "bouton_plus", "bouton_moins", et vérifie que compteur["valeur"] vaut 1.

7.2 Interaction client-serveur

DÉFINITION D3

Le **client** désigne le navigateur de l'utilisateur ; le **serveur** désigne la machine distante qui héberge le site Web. Une interaction Web typique suit un ordre précis : le client envoie une **requête**, le serveur la traite et renvoie une **réponse**, que le client affiche.

Propriété Ce qui s'exécute où. Le code exécuté sur le **serveur** (souvent en Python, PHP, etc.) génère la page à partir de données (base de données, calculs) : l'utilisateur ne voit jamais ce code. Le code exécuté sur le **client** (en JavaScript, dans le navigateur) réagit aux interactions de l'utilisateur **sans nouvel échange avec le serveur** (par exemple, afficher un message d'alerte), sauf s'il déclenche explicitement une nouvelle requête.

CODE DE RÉFÉRENCE — Modélisation de l'ordre client-serveur

```

1 etapes = []
2
3 def navigateur_envoie_requete(url):

```

```

4     etapes.append(("client", "envoie requete vers " + url))
5
6     def serveur_recoit_et_traite(url):
7         etapes.append(("serveur", "traite la requete " + url))
8         return "page HTML generee"
9
10    def serveur_revoie_reponse(contenu):
11        etapes.append(("serveur", "renvoie la reponse"))
12
13    def navigateur_affiche(contenu):
14        etapes.append(("client", "affiche : " + contenu))
15
16    navigateur_envoie_requete("/accueil")
17    contenu = serveur_recoit_et_traite("/accueil")
18    serveur_revoie_reponse(contenu)
19    navigateur_affiche(contenu)
20
21    print([acteur for acteur, action in etapes])
22    # ['client', 'serveur', 'serveur', 'client']

```

DÉFINITION D4

Un **cookie** est une petite donnée que le serveur demande au navigateur de **mémoriser** côté client. À chaque requête ultérieure vers le même site, le navigateur **retransmet** automatiquement ce cookie au serveur (par exemple, pour reconnaître un utilisateur déjà connecté).

CODE DE RÉFÉRENCE — Modélisation d'un cookie mémorisé côté client

```

1     cookies_client = {}
2
3     def serveur_definit_cookie(nom, valeur):
4         cookies_client[nom] = valeur
5
6     def client_envoie_requete_avec_cookies(url):
7         """Le client retransmet automatiquement les cookies memorises."""
8         return {"url": url, "cookies": dict(cookies_client)}
9
10    serveur_definit_cookie("id_session", "abc123")
11    requete = client_envoie_requete_avec_cookies("/panier")
12    print(requete) # {'url': '/panier', 'cookies': {'id_session': 'abc123'}}

```

Propriété Transmission chiffrée (HTTPS). Sans chiffrement (protocole **HTTP**), les données échangées entre client et serveur circulent **en clair** : n'importe quel intermédiaire sur le réseau peut les lire. Le protocole **HTTPS** chiffre ces échanges, rendant leur contenu illisible pour un intermédiaire. On utilise HTTPS dès que des informations sensibles (mot de passe, numéro de carte bancaire) sont transmises.

ERREUR FRÉQUENTE

Croire qu'un site en HTTPS est à l'abri de **tout** risque. HTTPS protège la **transmission** des données (personne ne peut les lire en chemin), mais ne garantit rien sur ce que le serveur **fait** des données une fois reçues (stockage sécurisé ou non, revente à des tiers, etc.).

» **Python À la machine.** Reprends le code du gestionnaire de cookies. Ajoute une fonction `supprimer_cookie(nom)` qui retire un cookie de `cookies_client` (par exemple lors d'une déconnexion). Vérifie qu'après suppression, une nouvelle requête ne contient plus ce cookie.

7.3 Formulaires, requêtes GET et POST

DÉFINITION D5

Un **formulaire** Web (balise `<form>`) regroupe des composants de saisie (champs de texte, cases à cocher...) et un bouton d'envoi. Lorsque l'utilisateur **soumet** le formulaire, le navigateur envoie une requête au serveur, contenant les valeurs saisies sous forme de **paramètres**.

Exemple. Un formulaire de recherche simple :

```
<form action="/recherche" method="GET">
  <input type="text" name="q">
  <button type="submit">Rechercher</button>
</form>
```

Propriété Requête GET. Avec la méthode GET, les paramètres du formulaire sont ajoutés **directement dans l'URL**, après un point d'interrogation, sous la forme `cle1=valeur1&cle2=valeur2`. Ils sont donc **visibles** (dans la barre d'adresse, dans l'historique du navigateur) et de **taille limitée**.

CODE DE RÉFÉRENCE — Construire une requête GET (module `urllib.parse`)

```
1 from urllib.parse import urlencode
2
3 parametres = {"q": "chaton", "page": 2}
4 url_get = "https://exemple.fr/recherche?" + urlencode(parametres)
5 print(url_get)
6 # https://exemple.fr/recherche?q=chaton&page=2
```

Propriété Requête POST. Avec la méthode POST, les paramètres sont placés dans le **corps** de la requête, pas dans l'URL : ils ne sont **pas visibles** dans la barre d'adresse et ne sont pas limités en taille de la même façon.

Exemple. Le formulaire de connexion suivant utilise POST, pour ne pas exposer le mot de passe dans l'URL :

```
<form action="/connexion" method="POST">
  <input type="text" name="identifiant">
```

```
<input type="password" name="mdp">
<button type="submit">Se connecter</button>
</form>
```

Propriété Choisir entre GET et POST. On choisit GET pour des paramètres **non sensibles** et courts (recherche, filtres, pagination), en profitant du fait qu'une URL GET peut être partagée, mise en favori ou revisitée facilement. On choisit POST pour des données **confidentielles** (mots de passe) ou **volumineuses** (envoi d'un fichier), qui ne doivent pas apparaître dans l'URL ni dans l'historique du navigateur.

ERREUR FRÉQUENTE

Utiliser GET pour transmettre un mot de passe ou toute donnée confidentielle. Même sur une connexion chiffrée (HTTPS), l'URL complète (avec les paramètres GET) reste visible dans **l'historique du navigateur** et peut être enregistrée dans les journaux (*logs*) du serveur : ce n'est pas un canal adapté à des données sensibles.

» **Python À la machine.** Écris une fonction `construire_url(base, parametres)` qui utilise `urlencode` pour construire une URL de requête GET à partir d'un dictionnaire de paramètres quelconque. Teste-la avec `ville: "Tunis", "tri": "prix"` : `ville: "Tunis", "tri": "prix"`.

Exercice 1

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 2

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction `basculer_affichage()` qui inverse la valeur de `etat["visible"]` (initialement `False`) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec `ajouter_gestionnaire`.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 3

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 4

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 5

15 min

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

QCM d'auto-évaluation — Interactions homme-machine sur le Web

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1/C2 — Composants et événements

1. [Q1] Le comportement d'un bouton lorsqu'on clique dessus est généralement programmé :
 - A. directement dans le code HTML.
 - B. en JavaScript, via un gestionnaire d'événement.

- C. dans la feuille de style CSS.
- D. il n'est jamais possible de programmer un tel comportement.

C4 — Client-serveur

2. [Q2] Le code exécuté **sur le serveur** :
- A. est visible dans le code source de la page reçue par le navigateur.
 - B. n'est jamais visible par l'utilisateur.
 - C. s'exécute toujours après le code JavaScript du client.
 - D. ne peut jamais accéder à une base de données.

C6 — Chiffrement

3. [Q3] Le protocole HTTPS garantit que :
- A. les données sont illisibles pour un intermédiaire pendant la transmission.
 - B. le serveur stocke les données de façon sécurisée.
 - C. aucune donnée personnelle n'est jamais collectée.
 - D. le site est fiable et digne de confiance.

C8 — GET et POST

4. [Q4] Les paramètres d'une requête POST sont :
- A. visibles dans l'URL de la page.
 - B. placés dans le corps de la requête, invisibles dans l'URL.
 - C. toujours chiffrés automatiquement.
 - D. limités à un seul paramètre.

C9 — Choix du type de requête

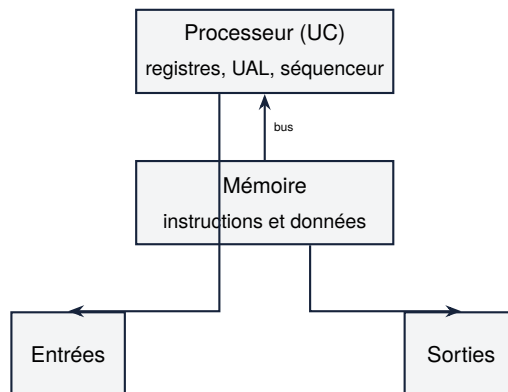
5. [Q5] Pour transmettre un mot de passe lors d'une connexion, il est recommandé d'utiliser :
- A. GET, car c'est la méthode la plus simple.
 - B. POST, pour ne pas exposer le mot de passe dans l'URL.
 - C. indifféremment GET ou POST, cela n'a aucune importance.
 - D. aucune des deux, un mot de passe ne doit jamais être transmis.
6. [Q6] Une recherche de produits sur un site marchand (mots-clés, filtres) utilise généralement :
- A. GET, car les critères ne sont pas confidentiels et l'URL peut être partagée.
 - B. POST, systématiquement, quelle que soit la nature des données.
 - C. ni GET ni POST : uniquement des cookies.
 - D. un formulaire sans méthode définie.

8 Architecture de von Neumann et systèmes d'exploitation

8.1 L'architecture de von Neumann

DÉFINITION D1

L'**architecture de von Neumann** décrit une machine composée de quatre constituants principaux, reliés par un **bus** : le **processeur** (ou UC, unité centrale), la **mémoire**, et les périphériques d'**entrée-sortie**.



Propriété Rôle de chaque constituant. Le **processeur** exécute les instructions, une par une : il lit une instruction en mémoire, la decode, l'exécute, puis passe à la suivante. La **mémoire** stocke à la fois les **instructions** du programme et les **données** qu'il manipule (c'est le principe clé de von Neumann : un même espace mémoire pour le code et les données). Les **entrées-sorties** permettent à la machine de communiquer avec l'extérieur (clavier, écran, disque, réseau).

DÉFINITION D2

À l'intérieur du processeur, les **registres** sont de petites mémoires très rapides qui stockent temporairement les valeurs en cours de traitement. L'**UAL** (unité arithmétique et logique) effectue les calculs (addition, comparaison...). Le **séquenceur** pilote l'enchaînement des étapes d'exécution.

Propriété Monoprocasseur et multiprocasseur. Une architecture **monoprocasseur** ne dispose que d'un seul processeur : les instructions s'exécutent **une à la fois**, séquentiellement. Une architecture **multiprocasseur** dispose de plusieurs processeurs (ou plusieurs *cœurs*), qui peuvent exécuter des instructions **en parallèle**.

ERREUR FRÉQUENTE

Croire que le processeur « sait » distinguer une instruction d'une donnée en mémoire. Dans l'architecture de von Neumann, instructions et données sont stockées dans le **même espace mémoire**, sous forme de suites de bits identiques dans leur nature : c'est uniquement le **contexte de lecture** (est-ce le séquenceur qui lit à cette adresse pour exécuter, ou l'UAL qui lit une donnée pour calculer ?) qui détermine comment ces bits sont interprétés.

» **Python À la machine.** Recherche le nombre de cœurs du processeur de l'ordinateur que tu utilises (par exemple avec la commande `nproc` sous Linux, ou dans les informations système). Ce processeur est-il monoprocesseur ou multiprocesseur au sens du cours ?

8.2 Dérouler l'exécution d'instructions machine

DÉFINITION D3

Une **instruction machine** est une opération élémentaire que le processeur sait exécuter directement : charger une valeur depuis la mémoire dans un registre, effectuer un calcul entre registres, écrire un registre en mémoire, etc. Un **programme machine** est une suite d'instructions, exécutées **dans l'ordre**, une par une.

Exemple. Un jeu d'instructions minimal, inspiré du langage machine :

- ▶ ("LOAD", registre, adresse) : charge la valeur de la mémoire à adresse dans registre.
- ▶ ("ADD", r1, r2) : ajoute le contenu de r2 à r1 (résultat dans r1).
- ▶ ("STORE", registre, adresse) : écrit le contenu de registre en mémoire à adresse.
- ▶ ("HALT",) : arrête l'exécution.

CODE DE RÉFÉRENCE — Simuler l'exécution d'un programme machine

```

1 def executer(programme, memoire):
2     """Exécute un programme machine simplifié, instruction par instruction."""
3     registres = {}
4     for instruction in programme:
5         op = instruction[0]
6         if op == "LOAD":
7             _, reg, adresse = instruction
8             registres[reg] = memoire[adresse]
9         elif op == "ADD":
10            _, reg_dest, reg_src = instruction
11            registres[reg_dest] = registres[reg_dest] + registres[reg_src]
12        elif op == "STORE":
13            _, reg, adresse = instruction
14            memoire[adresse] = registres[reg]
15        elif op == "HALT":
16            break
17    return memoire, registres
18
19 memoire = [5, 3, 0] # memoire[0]=5, memoire[1]=3, memoire[2]=0
20 programme = [
21     ("LOAD", "R0", 0), # R0 <- memoire[0] = 5
22     ("LOAD", "R1", 1), # R1 <- memoire[1] = 3
23     ("ADD", "R0", "R1"), # R0 <- R0 + R1 = 8
24     ("STORE", "R0", 2), # memoire[2] <- R0 = 8

```

```

25     ("HALT",),
26 ]
27 memoire_finale, registres = executer(programme, memoire)
28 print(memoire_finale)    # [5, 3, 8]
29 print(registres)         # {'R0': 8, 'R1': 3}

```

Propriété Dérouler une exécution. Dérouler l'exécution d'un programme consiste à indiquer, après **chaque** instruction, l'état des registres et de la mémoire à cet instant précis — c'est ce qui permet de vérifier ou de prédire le résultat d'un programme sans avoir besoin de l'exécuter sur une vraie machine.

ERREUR FRÉQUENTE

Oublier que les instructions s'exécutent **dans l'ordre d'écriture** du programme, une à la fois. Un déroulé d'exécution qui « saute » une instruction ou en anticipe une plus loin (sans branchement explicite) est incorrect.

» **Python À la machine.** Ajoute une instruction ("SUB", r1, r2) (soustraction : $r1 \leftarrow r1 - r2$) au simulateur executer. Écris un programme qui calcule `memoire[0] - memoire[1]` et le stocke dans `memoire[2]`, pour `memoire = [10, 4, 0]`.

8.3 Systèmes d'exploitation

DÉFINITION D4

Un **système d'exploitation** (OS, *operating system*) est le logiciel qui gère les ressources matérielles d'une machine (processeur, mémoire, disque, périphériques) et fournit une interface pour exécuter d'autres programmes.

Propriété Fonctions d'un système d'exploitation. Un système d'exploitation assure notamment : la **gestion des processus** (démarrer, arrêter, partager le temps processeur entre plusieurs programmes), la **gestion de la mémoire** (allouer de l'espace à chaque programme), la **gestion des fichiers** (organiser le stockage sur disque), et la **gestion des droits** (contrôler qui a le droit de faire quoi).

ERREUR FRÉQUENTE

Confondre un système d'exploitation avec une simple « interface graphique ». L'interface graphique (bureau, fenêtres, icônes) n'est qu'une des façons de présenter le système d'exploitation à l'utilisateur : le système d'exploitation lui-même fonctionne indépendamment de la présence ou non d'une interface graphique (un serveur peut tourner sans aucun affichage graphique).

Propriété Systèmes libres et propriétaires. Un système d'exploitation **libre** (comme Linux) met à disposition son code source, modifiable et redistribuable librement. Un système **propriétaire**

(comme Windows ou macOS) ne diffuse pas son code source. Les élèves de NSI utilisent généralement un système libre en travaux pratiques.

8.4 La ligne de commande

DÉFINITION D5

La **ligne de commande** (ou *terminal*, ou *shell*) permet de piloter le système d'exploitation en tapant des commandes textuelles, plutôt qu'en cliquant sur des icônes.

Propriété Commandes de base sur fichiers et dossiers.

ls	lister le contenu d'un dossier
cd chemin	se déplacer dans un dossier
mkdir nom	créer un dossier
touch fichier	créer un fichier vide
rm fichier	supprimer un fichier
cat fichier	afficher le contenu d'un fichier

CODE DE RÉFÉRENCE — Exécuter de vraies commandes shell depuis Python

```
1 import subprocess
2
3 subprocess.run(["mkdir", "dossier_test"])
4 subprocess.run(["touch", "dossier_test/notes.txt"])
5 resultat = subprocess.run(["ls", "dossier_test"], capture_output=True, text=True)
6 print(resultat.stdout) # notes.txt
```

8.5 Droits et permissions d'accès

DÉFINITION D6

Sous un système de type Unix (Linux, macOS), chaque fichier a trois catégories d'utilisateurs associées à des droits : le **propriétaire**, le **groupe**, et **les autres**. Pour chaque catégorie, trois droits peuvent être accordés ou refusés : **lecture** (r), **écriture** (w), **exécution** (x).

Exemple. La chaîne "rwxr-xr--" se lit par blocs de 3 caractères : rwx (propriétaire : lecture, écriture, exécution), r-x (groupe : lecture, exécution, pas d'écriture), r-- (autres : lecture seule).

CODE DE RÉFÉRENCE — Vérifier un droit d'accès à partir de sa représentation

```
1 def droit_accorde(permissions, categorie, action):
2     """Teste si 'action' est autorisée pour 'categorie' selon 'permissions'.

```

```

9     decalage = {"lecture": 0, "écriture": 1, "exécution": 2}
10    position = index[categorie] + decalage[action]
11    return permissions[position] != "-"
12
13    perm = "rwxr-xr--"
14    print(droit_accorde(perm, "proprietaire", "écriture")) # True
15    print(droit_accorde(perm, "autres", "écriture"))       # False

```

Propriété Notation octale. Chaque bloc de 3 droits (lecture=4, écriture=2, exécution=1) se résume en un **chiffre octal** égal à la somme des droits accordés. `rwX` vaut $4 + 2 + 1 = 7$, `r-x` vaut $4 + 0 + 1 = 5$, `r--` vaut 4 : on retrouve la notation `chmod 754` bien connue des utilisateurs de Linux.

» **Python À la machine.** Dans un terminal Linux, exécute la commande `ls -l` dans un dossier de ton choix : observe la colonne de permissions (par exemple `-rw-r--r--`). Identifie le propriétaire, le groupe, et les droits accordés aux autres utilisateurs pour un des fichiers listés.

Exercice 1 10 min

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 2 12 min

On reprend le simulateur `executer` du cours.

1. Ajouter à `executer` le traitement de l'instruction `("SUB", reg_dest, reg_src)`, qui effectue `reg_dest <- reg_dest - reg_src`.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```

1  memoire = [10, 4, 0]
2  programme = [
3      ("LOAD", "R0", 0),
4      ("LOAD", "R1", 1),
5      ("SUB", "R0", "R1"),
6      ("STORE", "R0", 2),
7      ("HALT",),
8  ]

```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 3 10 min

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?

2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 4 ♦♦

⌚ 10 min

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 5 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

QCM d'auto-évaluation — Architecture de von Neumann et systèmes d'exploitation

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Architecture de von Neumann

1. [Q1] Dans l'architecture de von Neumann, instructions et données sont stockées :
 - A. dans deux mémoires physiquement séparées.
 - B. dans le même espace mémoire.
 - C. uniquement dans les registres du processeur.
 - D. uniquement sur le disque dur.

C2 — Exécution d'instructions

2. [Q2] Les instructions d'un programme machine s'exécutent :
 - A. dans un ordre aléatoire.
 - B. toutes en même temps.
 - C. une par une, dans l'ordre du programme (sauf branchement).
 - D. dans l'ordre inverse du programme.

C3 — Systèmes d'exploitation

3. [Q3] La gestion des processus par le système d'exploitation permet notamment de :
 - A. donner l'illusion d'une exécution simultanée de plusieurs programmes sur un seul cœur.

- B. empêcher tout programme de s'exécuter.
- C. remplacer le processeur.
- D. afficher uniquement des fenêtres graphiques.

C4 — Ligne de commande

4. [Q4] La commande qui liste le contenu d'un dossier est :
- A. cd
 - B. ls
 - C. mkdir
 - D. rm

C5 — Droits et permissions

5. [Q5] Dans la chaîne de permissions `rw-r-x---`, le **groupe** peut :
- A. lire et écrire, mais pas exécuter.
 - B. lire et exécuter, mais pas écrire.
 - C. tout faire (lecture, écriture, exécution).
 - D. rien faire du tout.
6. [Q6] La notation octale `644` correspond aux permissions :
- A. `rw-r--r--`
 - B. `rw-rwxrwx`
 - C. `r--r--r--`
 - D. `rw-rw-rw-`

9 Réseaux : transmission de données

9.1 Découpage en paquets et encapsulation

DÉFINITION D1

Un **paquet** est un petit bloc de données, accompagné d'un **en-tête** contenant des informations nécessaires à son acheminement et à sa remise en ordre : numéro du paquet, nombre total de paquets, adresse d'origine, adresse de destination...

| Un fichier volumineux n'est jamais envoyé d'un bloc » sur Internet est découpé en petits morceaux, appelés paquets.

Propriété Intérêt du découpage en paquets. Découper un message en paquets permet : de faire circuler plusieurs communications **simultanément** sur le même réseau (partage de la bande passante) ; de faire emprunter à des paquets d'un même message des **chemins différents** si nécessaire ; de ne **retransmettre que les paquets perdus** en cas de problème, plutôt que le message entier.

CODE DE RÉFÉRENCE — Découper et réassembler un message en paquets

```
1 def decouper_en_paquets(message, taille_max):
2     """Decoupe 'message' en paquets de taille au plus 'taille_max'."""
3     nb_paquets = (len(message) + taille_max - 1) // taille_max
4     paquets = []
5     for i in range(nb_paquets):
6         donnees = message[i * taille_max:(i + 1) * taille_max]
7         paquets.append({"numero": i, "total": nb_paquets, "donnees": donnees})
8     return paquets
9
10 def reassembler(paquets):
11     """Reassemble un message a partir de ses paquets, quel que soit leur ordre d'
12     arrivee."""
13     paquets_tries = sorted(paquets, key=lambda p: p["numero"])
14     return "".join(p["donnees"] for p in paquets_tries)
15
16 message = "BONJOURNSI"
17 paquets = decouper_en_paquets(message, 4)
18 print(paquets)
19 print(reassembler(paquets)) # BONJOURNSI
```

Propriété Encapsulation. L'**encapsulation** consiste à ajouter successivement des en-têtes autour des données à mesure qu'elles descendent dans les couches réseau (données applicatives → en-tête de transport → en-tête réseau → en-tête de liaison), chaque couche n'interprétant que **son propre** en-tête, sans avoir besoin de connaître le contenu des couches supérieures.

Exemple. Peu importe le contenu du message (texte, image, vidéo), l'en-tête réseau qui indique l'adresse IP de destination a toujours le même format : c'est ce qui permet à un routeur d'acheminer n'importe quel type de données sans avoir à en comprendre le contenu.

CODE DE RÉFÉRENCE — Modéliser l'encapsulation par imbrication de dictionnaires

```

1 def encapsuler(donnees, en_tete_transport, en_tete_reseau):
2     """Encapsule des donnees dans deux en-tetes successifs (transport puis reseau).
3     """
4     paquet_transport = {"en_tete": en_tete_transport, "donnees": donnees}
5     paquet_reseau = {"en_tete": en_tete_reseau, "donnees": paquet_transport}
6     return paquet_reseau
7
8 paquet = encapsuler("Bonjour", {"port": 443}, {"ip_dest": "192.168.1.10"})
9 print(paquet)
10 # le routeur ne lit que paquet["en_tete"] (ip_dest), sans decoder paquet["donnees"]

```

ERREUR FRÉQUENTE

Croire que les paquets d'un même message arrivent toujours **dans l'ordre** d'envoi. Le numéro de paquet dans l'en-tête sert précisément à les **réordonner** à l'arrivée : sans lui, un réassemblage naïf (dans l'ordre de réception) produirait un message incorrect si des paquets arrivent dans le désordre.

» **Python À la machine.** Modifie reassembler pour qu'elle lève une erreur explicite (assert) si le nombre de paquets reçus ne correspond pas au total indiqué dans leurs en-têtes (paquet manquant).

9.2 Le protocole du bit alterné

DÉFINITION D2

Le protocole du **bit alterné** est un mécanisme simple de récupération de perte de paquets. L'émetteur associe à chaque paquet un **bit** (0 ou 1) qui **alterne** à chaque nouveau paquet. Le récepteur renvoie un **accusé de réception** (ACK) contenant ce même bit. Si l'émetteur ne reçoit **pas** l'ACK attendu (paquet ou ACK perdu), il **retransmet** le paquet avec le même bit, jusqu'à recevoir l'ACK correct.

Propriété Pourquoi un simple bit suffit. Comme les paquets sont envoyés **un par un** (l'émetteur attend l'ACK avant d'envoyer le suivant), un seul bit alterné suffit à distinguer « un nouveau paquet » d'« une retransmission de l'ancien » : le récepteur sait qu'il a déjà reçu un paquet avec un bit donné, et ignore les doublons.

CODE DE RÉFÉRENCE — Simulation du protocole du bit alterné

```

1 def protocole_bit_alterne(messages, tentatives_perdues):
2     """Simule l'envoi de 'messages' avec le protocole du bit alterne.
3
4     'tentatives_perdues' : ensemble des numeros de tentatives (globales, a partir
5     de 1)
6     qui echouent (paquet ou ACK perdu), pour simuler des pertes reseau.
7     Renvoie la liste des (bit, message) effectivement recus par le destinataire.
8     """

```



```

8   resultat = []
9   bit = 0
10  compteur_tentative = 0
11  for message in messages:
12      recu = False
13      while not recu:
14          compteur_tentative += 1
15          if compteur_tentative in tentatives_perdues:
16              continue # echec : on retransmet le meme message avec le meme bit
17          resultat.append((bit, message))
18          recu = True
19          bit = 1 - bit # on alterne le bit pour le message suivant
20  return resultat
21
22  # La 2e tentative globale echoue : le premier envoi de "B" est perdu, il est
    retransmis
23  envois = protocole_bit_alterne(["A", "B", "C"], tentatives_perdues={2})
24  print(envois) # [(0, 'A'), (1, 'B'), (0, 'C')]

```

Propriété Robustesse du protocole. Quel que soit le nombre de pertes simulées, tant que le réseau finit par transmettre correctement (pas de perte **permanente**), tous les messages finissent par arriver : le protocole du bit alterné garantit la **livraison**, au prix éventuel de retransmissions et donc d'un délai supplémentaire.

9.3 Simuler un réseau

DÉFINITION D3

On peut modéliser un **réseau** local par un **graphe** : chaque machine est un sommet, chaque connexion directe (câble, Wi-Fi) est une arête. Deux machines peuvent communiquer si et seulement s'il existe un **chemin** entre elles dans ce graphe (éventuellement via des intermédiaires comme un commutateur ou un routeur).

CODE DE RÉFÉRENCE — Simuler un réseau et tester la communication entre deux machines

```

1  reseau = {
2      "PC1": ["Switch"],
3      "PC2": ["Switch"],
4      "Switch": ["PC1", "PC2", "Routeur"],
5      "Routeur": ["Switch", "Internet"],
6      "Internet": ["Routeur"],
7  }
8
9  def peuvent_communiquer(reseau, depart, arrivee, visites=None):
10     """Teste s'il existe un chemin entre 'depart' et 'arrivee' dans le reseau."""
11     if visites is None:
12         visites = set()
13     if depart == arrivee:
14         return True
15     visites.add(depart)
16     for voisin in reseau.get(depart, []):
17         if voisin not in visites:

```

```

18         if peuvent_communiquer(reseau, voisin, arrivee, visites):
19             return True
20     return False
21
22 print(peuvent_communiquer(reseau, "PC1", "Internet")) # True

```

ERREUR FRÉQUENTE

Oublier de marquer les sommets **déjà visités** lors du parcours d'un réseau. Sans cette précaution, un réseau contenant un cycle (par exemple deux commutateurs reliés entre eux et à d'autres machines) provoquerait une **boucle infinie** lors de la recherche de chemin.

» **Python À la machine.** Ajoute au réseau simulé une machine "PC3" reliée uniquement à "PC1" (et non au Switch). Vérifie que PC3 peut communiquer avec PC1 mais pas directement avec Internet.

9.4 Capteurs, actionneurs et IHM programmée

DÉFINITION D4

Un **capteur** mesure une grandeur physique (température, luminosité, présence...) et la convertit en une donnée numérique exploitable par un programme. Un **actionneur** reçoit une commande d'un programme et agit sur le monde physique (allumer une lampe, ouvrir un volet, déclencher un moteur...).

Exemple. Un thermostat connecté : le **capteur** de température mesure la température ambiante ; le programme décide, selon cette mesure, d'activer ou non le chauffage ; l'**actionneur** (relais électrique) coupe ou active effectivement le chauffage.

Propriété Cahier des charges d'une IHM. Réaliser une IHM (interface homme-machine) répondant à un **cahier des charges** consiste à traduire des règles énoncées en langage courant (« si la température descend sous 18°C, le chauffage s'active ») en un programme précis, testé sur des cas concrets.

CODE DE RÉFÉRENCE — Thermostat programmé : capteur, décision, actionneur

```

1  etat = {"chauffage": False, "override_manuel": None}
2
3  def decider_chauffage(temperature, seuil=18):
4      """Cahier des charges : le chauffage s'active si temperature < seuil,
5      sauf si un override manuel est actif (bouton de la telecommande)."""
6      if etat["override_manuel"] is not None:
7          return etat["override_manuel"]
8      return temperature < seuil
9
10 def actionner_chauffage(temperature, seuil=18):
11     """Lit le capteur (simule par le parametre temperature), decide,
12     et pilote l'actionneur (simule par etat["chauffage"])."""
13     etat["chauffage"] = decider_chauffage(temperature, seuil)
14     return etat["chauffage"]

```

```

15
16 def bouton_override(valeur):
17     """Gestionnaire d'événement du bouton manuel de la télécommande."""
18     etat["override_manuel"] = valeur
19
20 print(actionner_chauffage(15)) # True : il fait froid, chauffage active
21 print(actionner_chauffage(22)) # False : il fait assez chaud

```

Exemple. Le cahier des charges précise aussi qu'un utilisateur peut forcer manuellement le chauffage (bouton « override »), quelle que soit la température :

ERREUR FRÉQUENTE

Programmer une IHM en oubliant de tester les **cas limites** du cahier des charges (température exactement égale au seuil, override activé puis désactivé). Un cahier des charges mal testé peut sembler fonctionner sur l'exemple de démonstration tout en étant incorrect dans des situations réelles fréquentes.

» **Python À la machine.** Modifie le thermostat pour ajouter une **hystérésis** : le chauffage s'active si la température descend sous 18 °C, mais ne se désactive que lorsqu'elle dépasse 20 °C (pour éviter des allumages/extinctions trop fréquents). Teste ta fonction sur une séquence de températures [15, 17, 19, 21, 19].

Exercice 1

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (donnees) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par reassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 2

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 3

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :

```

1 reseau = {
2     "Salle1": ["Switch1"],
3     "Salle2": ["Switch1"],
4     "Switch1": ["Salle1", "Salle2", "Switch2"],

```

```
5     "Switch2": ["Switch1", "Salle3"],
6     "Salle3": ["Switch2"],
7 }
```

1. En utilisant `peuvent_communiquer` du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.
3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 4

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction `decider_eclairage(luminosite, seuil=300)` qui renvoie `True` si le lampadaire doit s'allumer.
3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 5

15 min

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction `irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60)` qui met à jour et renvoie l'état `etat["pompe"]` (initialement `False`), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures [20, 35, 50, 65, 40] (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

QCM d'auto-évaluation — Réseaux : transmission de données

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Paquets et encapsulation

1. [Q1] Dans l'en-tête d'un paquet, le numéro de paquet sert principalement à :
 - A. chiffrer les données transmises.
 - B. réordonner les paquets à l'arrivée, quel que soit leur ordre de réception.
 - C. accélérer la transmission.
 - D. indiquer la taille du disque du destinataire.

C2 — Protocole du bit alterné

2. [Q2] Si l'émetteur ne reçoit pas l'accusé de réception attendu, il :
- A. abandonne définitivement l'envoi du message.
 - B. retransmet le paquet avec le même bit.
 - C. retransmet le paquet avec le bit opposé.
 - D. envoie directement le paquet suivant.

C3 — Simulation de réseau

3. [Q3] Pour éviter qu'une recherche de chemin dans un réseau contenant un cycle ne boucle indéfiniment, il faut :
- A. interdire tout cycle dans le réseau.
 - B. marquer les sommets déjà visités.
 - C. limiter le réseau à deux machines.
 - D. ne jamais utiliser de fonction récursive.

C4 — Capteurs et actionneurs

4. [Q4] Un capteur de température et un relais de chauffage jouent respectivement les rôles de :
- A. actionneur puis capteur.
 - B. capteur puis actionneur.
 - C. deux capteurs.
 - D. deux actionneurs.

C5 — IHM programmée

5. [Q5] Programmer une IHM à partir d'un cahier des charges nécessite en particulier de :
- A. tester uniquement le cas de démonstration prévu.
 - B. tester aussi les cas limites (valeurs au seuil, changements d'état).
 - C. éviter tout test, le code étant supposé correct.
 - D. ignorer les exigences non précisées explicitement.
6. [Q6] Le principe d'hystérésis (deux seuils différents pour activer/désactiver un actionneur) sert à :
- A. rendre le système plus lent.
 - B. éviter des activations/désactivations trop fréquentes autour d'un seul seuil.
 - C. économiser de la mémoire.
 - D. remplacer le capteur par un actionneur.

10 Projet et méthodes

10.1 Repères d'histoire de l'informatique

DÉFINITION D1

L'histoire de l'informatique retrace comment les concepts d'**algorithme**, de **machine**, de **langage** et de **données** — les quatre notions fondamentales du programme — ont émergé et se sont articulés au fil du temps, à travers des personnes précises.

Propriété Quelques repères clés.

- ▶ Les **algorithmes** sont présents dès l'Antiquité (par exemple l'algorithme d'Euclide).
- ▶ Les premières **machines à calculer** apparaissent progressivement au XVII^e siècle.
- ▶ En **1936**, Alan Turing introduit le concept de **machine universelle** : une machine capable, en principe, d'exécuter n'importe quel algorithme — c'est la naissance de l'idée moderne d'ordinateur.
- ▶ Les **premiers ordinateurs** sont construits en **1948**.
- ▶ Le réseau **Internet** se développe depuis **1969**.

CODE DE RÉFÉRENCE — Manipuler une chronologie avec des tableaux de p-uplets

```
1  evenements = [  
2      (1936, "Concept de machine universelle", "Alan Turing"),  
3      (1948, "Premier ordinateur a programme enregistré", "Equipe de Manchester"),  
4      (1969, "Debuts du reseau Internet (ARPANET)", "DARPA"),  
5      (1971, "Premier microprocesseur commercial", "Intel"),  
6      (1989, "Invention du World Wide Web", "Tim Berners-Lee"),  
7      (1991, "Premiere version publique de Python", "Guido van Rossum"),  
8  ]  
9  
10 def evenements_entre(evenements, debut, fin):  
11     """Renvoie les evenements dont l'annee est comprise entre debut et fin (inclus  
12     )."""  
13     return [e for e in evenements if debut ≤ e[0] ≤ fin]  
14  
15 for annee, fait, personne in evenements_entre(evenements, 1940, 1970):  
16     print(annee, "-", fait, "(" + personne + ")")
```

ERREUR FRÉQUENTE

Réduire l'histoire de l'informatique à une suite de « dates à retenir », sans les relier aux concepts. Le programme précise que ces repères doivent être **construits progressivement**, en lien avec les notions techniques déjà étudiées : par exemple, 1936 (Turing) prend tout son sens une fois qu'on a compris la notion d'algorithme et de machine universelle du chapitre sur l'architecture.

» **Python À la machine.** Ajoute à la liste evenements deux événements de ton choix (recherchés dans une source fiable), avec leur année et leur protagoniste. Vérifie que la liste reste triée chronologiquement avec sorted.

10.2 La démarche de projet

DÉFINITION D2

Un **cahier des charges** décrit, avant de commencer à programmer, ce que le projet doit accomplir : les fonctionnalités attendues, les contraintes, et les critères qui permettront de dire si le projet est réussi.

Propriété Découper un projet en jalons. Un **jalon** (ou *point d'étape*) est une étape intermédiaire d'un projet, avec un objectif précis et vérifiable. Découper un projet en jalons permet de suivre sa progression, d'ajuster ses objectifs si nécessaire, et d'éviter de se retrouver bloqué en fin de projet.

CODE DE RÉFÉRENCE — Suivre l'avancement d'un projet par ses jalons

```

1 jalons = [
2     {"nom": "Cahier des charges", "termine": True},
3     {"nom": "Prototype minimal", "termine": True},
4     {"nom": "Fonctionnalites completes", "termine": False},
5     {"nom": "Tests et correction de bugs", "termine": False},
6     {"nom": "Presentation finale", "termine": False},
7 ]
8
9 def avancement(jalons):
10     """Renvoie le pourcentage de jalons termines."""
11     nb_terminees = sum(1 for j in jalons if j["termine"])
12     return nb_terminees / len(jalons) * 100
13
14 def prochain_jalon(jalons):
15     """Renvoie le nom du premier jalon non termine, ou None si tout est fini."""
16     for j in jalons:
17         if not j["termine"]:
18             return j["nom"]
19     return None
20
21 print(avancement(jalons))      # 40.0
22 print(prochain_jalon(jalons)) # Fonctionnalites completes

```

Propriété Travailler en équipe. Le programme officiel préconise des groupes de 2 à 4 élèves. Un projet en équipe suppose de **répartir les tâches** (par exemple par fonctionnalité, ou par couche : données, algorithmes, interface), de **coordonner le travail** (éviter que deux personnes modifient la même partie en même temps), et de tenir des **points d'étape** réguliers avec le professeur.

Le programme o
réserve au moins
quart de l'horaire
de Première à la
conception et à
l'élaboration de pr
menés par groupe
à 4 élèves.

ERREUR FRÉQUENTE

Vouloir réaliser un projet trop ambitieux, sans jalons intermédiaires. Le programme officiel insiste explicitement sur le fait que les projets doivent rester « d’une ambition raisonnable » pour permettre à l’équipe de les mener à terme : mieux vaut un projet modeste **complet** et testé, qu’un projet ambitieux inachevé.

» **Python À la machine.** Choisis un projet informatique simple de ton choix (par exemple un petit jeu, un gestionnaire de liste de tâches). Découpe-le en 4 jalons avec la structure jalons du cours, et calcule ton avancement au fur et à mesure que tu réalises chaque étape.

10.3 Déboguer méthodiquement

DÉFINITION D3

Déboguer un programme, c’est identifier puis corriger la cause d’un comportement incorrect (plantage, résultat faux). Une démarche **méthodique** vaut mieux qu’une correction « au hasard ».

Exemple. Le programme suivant plante lorsqu’on l’exécute :

```
1 def moyenne_ponderee_bugue(valeurs, poids):
2     total = 0
3     for i in range(len(valeurs)):
4         total += valeurs[i] * poids[i]
5     return total / sum(poids)
6
7 moyenne_ponderee_bugue([12, 15, 18], [1, 2])
```

```
Traceback (most recent call last):
  File "programme.py", line 7, in <module>
    moyenne_ponderee_bugue([12, 15, 18], [1, 2])
  File "programme.py", line 4, in moyenne_ponderee_bugue
    total += valeurs[i] * poids[i]
                ~~~~~^
IndexError: list index out of range
```

Propriété Lire un message d’erreur (traceback). Un **traceback** Python indique, de haut en bas, la **pile des appels** qui a mené à l’erreur, et se termine par le **type** de l’erreur (`IndexError`) et un **message** explicatif. La **dernière ligne de code** indiquée (ici, `total += valeurs[i] * poids[i]`) est l’endroit précis où l’erreur s’est produite — c’est le premier endroit à examiner.

Propriété Démarche méthodique de débogage.

1. Lire **entièrement** le message d’erreur : type d’erreur, ligne concernée.
2. Identifier la **cause** : ici, `poids` a seulement 2 éléments, mais la boucle tente d’accéder à `poids[2]` (car `valeurs` en a 3).
3. Formuler une **hypothèse** : les deux listes doivent avoir la même longueur.
4. **Vérifier** l’hypothèse, par exemple avec un `print(len(valeurs), len(poids))` ou une précondition `assert`.

5. Corriger : ajouter une précondition, ou corriger l'appel fautif.

CODE DE RÉFÉRENCE — Corriger avec une précondition explicite

```
1 def moyenne_ponderee(valeurs, poids):
2     assert len(valeurs) == len(poids), "valeurs et poids doivent avoir la meme
   longueur"
3     total = 0
4     for i in range(len(valeurs)):
5         total += valeurs[i] * poids[i]
6     return total / sum(poids)
```

Propriété Rôle du print et de assert en débogage. Insérer temporairement des `print(...)` pour afficher l'état de variables clés à des points précis du programme est une technique simple et efficace pour localiser un bug. Les `assert` permettent, eux, de **documenter** et **vérifier automatiquement** des hypothèses sur le programme, à chaque exécution.

ERREUR FRÉQUENTE

Corriger un bug en modifiant du code **au hasard**, sans avoir d'abord compris sa cause exacte. Cette approche peut sembler « résoudre » le problème par chance sur un cas, tout en laissant le bug intact (ou en en créant un nouveau) dans d'autres cas.

» **Python À la machine.** Provoque volontairement une `TypeError` en appelant `moyenne_ponderee(["a", "b"], [1, 2])` (des chaînes au lieu de nombres). Lis le traceback obtenu, identifie la ligne et le type d'erreur, puis explique en une phrase la cause du problème.

10.4 Documenter son code et préparer une présentation orale

DÉFINITION D4

Documenter un programme, c'est expliquer, en langage naturel, ce qu'il fait : le rôle de chaque fonction (via une **docstring**), les choix de conception importants, et la manière de l'utiliser.

CODE DE RÉFÉRENCE — Une fonction bien documentée

```
1 def calcul_moyenne(valeurs):
2     """Calcule la moyenne arithmétique d'une liste de valeurs numeriques.
3
4     Precondition : valeurs est une liste non vide.
5     Postcondition : le resultat est compris entre min(valeurs) et max(valeurs).
6     """
7     return sum(valeurs) / len(valeurs)
8
9 def a_une_docstring(fonction):
10     """Teste si 'fonction' possède une docstring non vide."""
11     return fonction.__doc__ is not None and len(fonction.__doc__.strip()) > 0
```

```
12
13 print(a_une_docstring(calcul_moyenne)) # True
```

Propriété Ce qu'une bonne documentation contient. Une docstring de fonction utile précise, dans l'ordre : **ce que fait** la fonction (une phrase claire), ses **préconditions** (ce que les arguments doivent vérifier), et éventuellement ses **postconditions** (ce que garantit le résultat) — exactement la spécification vue au chapitre sur les langages et la programmation.

Propriété Préparer une présentation orale de projet. Le programme officiel souligne que cette spécialité contribue au développement des **compétences orales**, en particulier pour préparer l'épreuve orale de terminale (Grand Oral) — un travail qui commence dès la Première. Une présentation de projet efficace explique : le **problème** traité, les **choix** effectués (et pourquoi), les **difficultés** rencontrées et comment elles ont été surmontées, et une **démonstration** concrète du résultat.

ERREUR FRÉQUENTE

Préparer une présentation orale qui ne fait que **décrire le code ligne par ligne**. Un jury (ou un public) s'intéresse davantage au **raisonnement** : pourquoi ce problème est intéressant, pourquoi telle approche a été choisie plutôt qu'une autre, ce qui a été appris en le résolvant — pas à une lecture littérale du code source.

» **Python À la machine.** Choisis une fonction que tu as écrite dans un chapitre précédent de ce manuel (par exemple le tri par insertion ou la recherche dichotomique). Écris-lui une docstring complète (rôle, précondition, postcondition), puis vérifie avec `a_une_docstring` qu'elle est bien documentée. Prépare ensuite, à l'oral, une explication de 2 minutes de cette fonction pour un camarade qui ne l'a jamais vue.

Exercice 1 10 min

On donne la chronologie suivante :

```
1 evenements = [
2     (1642, "Pascaline, machine a calculer mecanique", "Blaise Pascal"),
3     (1936, "Concept de machine universelle", "Alan Turing"),
4     (1971, "Premier microprocesseur commercial", "Intel"),
5     (1991, "Premiere version publique de Python", "Guido van Rossum"),
6 ]
```

1. Vérifier, avec `sorted`, que cette liste est bien dans l'ordre chronologique.
2. En utilisant `evenements_entre` du cours, lister les événements situés entre 1900 et 1980.
3. Situe brièvement, en une phrase chacun, ce qu'apporte historiquement le concept de « machine universelle » de Turing (1936) par rapport à la Pascaline (1642).

Exercice 2 10 min

Une équipe de 3 élèves développe un petit jeu vidéo, découpé en 6 jalons :

```
1 jalons = [
2     {"nom": "Specification du jeu", "termine": True},
```

```

3     {"nom": "Déplacement du personnage", "termine": True},
4     {"nom": "Gestion des collisions", "termine": True},
5     {"nom": "Score et niveaux", "termine": False},
6     {"nom": "Tests et équilibrage", "termine": False},
7     {"nom": "Présentation finale", "termine": False},
8 ]

```

1. En utilisant `avancement` du cours, calculer le pourcentage d'avancement du projet.
2. En utilisant `prochain_jalon` du cours, déterminer sur quel jalon l'équipe doit désormais se concentrer.
3. L'équipe est à mi-parcours de l'horaire prévu pour ce projet, et son avancement est justement de 50 %. Les 3 jalons restants (score/niveaux, tests, présentation) te semblent-ils du même volume de travail que les 3 jalons déjà réalisés ? Pourquoi ce simple pourcentage peut-il être trompeur ?

Exercice 3 ♦♦

Le programme suivant plante à l'exécution :

```

1 notes = {"Alice": 15, "Bob": 12, "Chloe": 18}
2
3 def afficher_note(notes, nom):
4     print(nom, ":", notes[nom])
5
6 afficher_note(notes, "David")

```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En suivant la démarche méthodique du cours, identifier précisément la cause du problème (sans se contenter de « il y a une erreur »).
3. Proposer une correction de `afficher_note` qui affiche un message clair (par exemple "Nom inconnu") au lieu de planter, lorsque le nom n'existe pas dans `notes`.

Exercice 4 ♦♦

On reprend la fonction `tri_insertion` d'un chapitre précédent, sans aucune documentation :

```

1 def tri_insertion(tableau):
2     tableau = tableau[:]
3     for i in range(1, len(tableau)):
4         valeur = tableau[i]
5         j = i - 1
6         while j ≥ 0 and tableau[j] > valeur:
7             tableau[j + 1] = tableau[j]
8             j -= 1
9         tableau[j + 1] = valeur
10    return tableau

```

1. Vérifier, avec `a_une_docstring` du cours, que cette fonction n'est pas documentée.
2. Écrire une docstring complète pour cette fonction (rôle, précondition, postcondition), puis vérifier qu'elle est bien détectée comme documentée.
3. Si tu devais présenter cette fonction en 2 minutes à l'oral, quels seraient les 3 points les plus importants à expliquer (au-delà d'une lecture ligne par ligne du code) ?

Exercice 5 ◆◆◆

Une équipe développe un gestionnaire de tournoi. La veille de la présentation finale (dernier jalon du projet), un testeur signale que la fonction `rechercher_participant` ne trouve jamais le **dernier** inscrit de la liste :

```
1 def rechercher_participant_bugue(participants, pseudo):
2     for i in range(len(participants) - 1):
3         if participants[i] == pseudo:
4             return i
5     return -1
6
7 participants = ["Ali", "Sara", "Nora", "Yanis"]
8 print(rechercher_participant_bugue(participants, "Yanis"))
```

1. Que renvoie cet appel ? Explique précisément, en examinant la boucle `for`, pourquoi le dernier élément de `participants` n'est jamais examiné.
2. Corriger la fonction.
3. L'équipe est à la veille de la présentation, avec ce bug encore présent dans une fonctionnalité secondaire. En te référant à la démarche de projet du cours (jalons, ambition raisonnable), quelle serait une décision d'équipe raisonnable : corriger dans l'urgence juste avant de présenter, ou documenter le bug comme limite connue et le corriger après la présentation ? Justifie ta réponse en une ou deux phrases.

QCM d'auto-évaluation — Projet et méthodes

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Histoire de l'informatique

1. [Q1] Le concept de machine universelle, capable en principe d'exécuter n'importe quel algorithme, est introduit par :
 - A. Blaise Pascal en 1642.
 - B. Alan Turing en 1936.
 - C. Guido van Rossum en 1991.
 - D. Tim Berners-Lee en 1989.

C2 — Démarche de projet

2. [Q2] Le programme officiel réserve à la conception de projets, en classe de Première :
 - A. moins d'un dixième de l'horaire annuel.
 - B. au moins un quart de l'horaire annuel.
 - C. la totalité de l'horaire annuel.
 - D. aucune heure dédiée.
3. [Q3] Découper un projet en jalons permet notamment de :
 - A. éviter tout travail en équipe.
 - B. suivre la progression et ajuster les objectifs si nécessaire.
 - C. garantir qu'aucun bug ne subsistera.
 - D. remplacer le cahier des charges.

C3 — Débogage

4. [Q4] Face à un message d'erreur (traceback) Python, la première chose à faire est de :
- A. corriger au hasard une ligne du programme.
 - B. lire entièrement le message pour identifier le type d'erreur et la ligne concernée.
 - C. ignorer le message et relancer le programme.
 - D. supprimer toute la fonction concernée.

C4 — Documentation et oral

5. [Q5] Une bonne docstring de fonction précise notamment :
- A. le nom de l'auteur du code uniquement.
 - B. le rôle de la fonction, ses préconditions et ses postconditions.
 - C. la date de la dernière modification uniquement.
 - D. rien : le code se suffit toujours à lui-même.
6. [Q6] Une présentation orale efficace d'un projet informatique privilégie :
- A. la lecture du code source ligne par ligne.
 - B. le raisonnement : problème traité, choix effectués, difficultés surmontées.
 - C. l'énumération de tous les noms de variables utilisées.
 - D. la récitation de la documentation officielle de Python.